UNIVERSITAT POLITÈCNICA
DE CATALUNYA
BARCELONATECH

Manresa School of Engineering

From Passwords to Passkeys

Design, Implementation and Security Evaluation of WebAuthn-Based Passwordless Authentication

Manresa, May 8, 2026

Master's Thesis (18 ECTS) submitted by

Jordi Nadeu Ferran

in partial fulfillment of the requirements for the

Master's degree in Machine Learning and Cybersecurity for Internet Connected Systems

Advisor: Aleix Llusà Serra

Topics: ICT, cybersecurity, WebAuthn, passkeys, W3C, FIDO2

This work is licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 License. To view a copy of this license, visit `https://creativecommons.org/licenses/by-nc-sa/4.0/legalcode` or send a letter to Creative Commons, PO Box 1866, Mountain View, CA 94042, USA.

This manuscript has been typeset using LaTeX and the class `tfgitic` version 8.2

Declaration of Responsibility

I, Jordi Nadeu Ferran, author of this Master's Thesis,

DECLARE

That this project and the accompanying report are original and solely the result of my work. Furthermore, all sources consulted have been included in the bibliography.

By submitting this report for evaluation, I consider this declaration signed for the purposes outlined in the agreement CG/2019/05/10, dated October 8, 2019, of the Governing Council of the Universitat Politècnica de Catalunya, which approves the procedure for plagiarism prevention.

Manresa, May 8, 2026

Acknowledgments

I would like to express my sincere gratitude to my director for his guidance and constructive feedback throughout this project.

I would also like to thank the W3C WebAuthn Working Group and the FIDO Alliance for their open and meticulously documented standards, without which this project would not be possible.

Finally, I would like to thank my colleagues and my family, whose patience and support have been essential in enabling me to complete this work

Abstract

The password based authentication remains the predominant credential model on the web, despite the well documented vulnerabilities to phishing, large scale database breaches and credential reuse.

This paradigm is changing today, because the FIDO 2 open standard, provides a public key cryptographic alternative that is architecturally phishing resistant.

The WebAuthn Level 2 introduced a cryptographic alternative based on hardware bound public key credentials, but the requirement that each device store a non-exportable private key created an account recovery problem that limited user adoption.

For it's part, the WebAuthn Level 3 extends the specification with synchronised *passkeys*, that are discoverable public key credentials which private key is replicated across all of the user's devices via a platform account such as iCloud Keychain, Google Password Manager or Microsoft Account.

The passkeys addresses the recovery gap but it makes the security of every passkey is dependent by the weakest link in the platform account authentication chain, this platform dependency introduces a new attack vectors that the relying parties cannot observes.

This work investigates if synchronised passkeys preserve the cryptographic security properties of the hardware bound model and what new trade-offs the synchronisation architecture introduces.

The study combines an analysis of the WebAuthn Level 3 changes with the design and implementation of a passkey prototype login that performs the registration and authentication ceremonies directly against the specification.

The tests suite of the prototype validate the security properties and confirms that synchronised passkeys preserve phishing resistance, replay prevention, man-in-the-middle attacker resistance and the elimination of server side shared secrets.

Also we perform a comparative evaluation of the principal web authentication mechanisms, such as passwords, one time password, federated login, multifactor authentication, WebAuthn Level 2 hardware and WebAuthn Level 3 passkey, according to sixteen criteria grouped into security, usability, deployment and privacy categories.

The clone detection, available in the hardware bound model, becomes structurally unavailable for synchronised credentials.

In conclusions, we can affirm that in the most consumer and enterprise contexts, the passkeys represents a clear security improvement compared to conventional mechanisms.

Resum

Les contrasenyes com a mètode d'autenticació continua sent el model de credencial predominant a la web, malgrat les vulnerabilitats ben documentades al phishing, les filtracions de bases de dades a gran escala i la reutilització de credencials.

Aquest paradigma està canviant avui dia, perquè l'estàndard obert FIDO 2 proporciona una alternativa criptogràfica de clau pública que és arquitectònicament resistent al phishing.

El WebAuthn nivell 2 va introduir una alternativa criptogràfica basada en credencials de clau pública lligades al dispositiu (hardware), però el requisit que cada dispositiu emmagatzemés una clau privada no exportable va crear un problema de recuperació de compte que va limitar l'adopció per part dels usuaris.

D'altre banda, el WebAuthn nivell 3 amplia l'especificació del estàndard i proposa un nou concepte anomenat *passkeys*, que són credencials de clau pública descobribles de les quals la clau privada es replica a tots els dispositius de l'usuari a través d'un compte de plataforma com ara iCloud Keychain, Google Password Manager o Microsoft Account.

Les *passkeys* aborden la bretxa que suposava el procés de recuperació de les claus, però fan que la seguretat de cada passkey depengui de l'enllaç més feble de la cadena d'autenticació de comptes de la plataforma, aquesta dependència de la plataforma introdueix nous vectors d'atac que les *relying parties* no poden observar.

Aquest treball investiga si les *passkeys* sincronitzades preserven les propietats de seguretat criptogràfica del model lligat al dispositiu (hardware) i quins nous reptes introdueix l'arquitectura de sincronització.

El treball combina una anàlisi dels canvis del WebAuthn nivell 3 amb el disseny i la implementació d'un prototip d'inici de sessió amb passkeys que realitza les cerimònies de registre i autenticació directament segons l'especificació.

Després una suite de proves del prototip valida les propietats de seguretat i confirma que les passkeys sincronitzades preserven la resistència al phishing, la prevenció de la reutilització, la resistència a l'atac MITM i l'eliminació dels secrets compartits del costat del servidor.

També es realitza una avaluació comparativa sobre els principals mecanismes d'autenticació web, com són les contrasenyes, OTPs, inici de sessió federats, autenticació multifactorial (MFA), dispositius WebAuthn nivell 2 i les *passkey* WebAuthn nivell 3, segons setze criteris agrupats en les categories de seguretat, usabilitat, desplegament i privacitat.

La detecció de clons, disponible en el model vinculat al dispositiu, esdevé estructuralment inviable per a les credencials sincronitzades (*passkeys*).

En conclusió, podem afirmar que en la gran majoria de contextos d'usuaris particulars i també empresarials, les *passkeys* representen una millora clara en la seguretat en comparació amb els mecanismes convencionals.

Contents

Abstract iv

Resum iv

1 Introduction 1
1.1 Motivation . . . 1
1.2 Research Objectives . . . 2
1.3 Research Questions . . . 3
1.4 Scope and Limitations . . . 3

2 Academic Research about Web Authentication 4
2.1 Prior Literature . . . 4
2.2 Taxonomy of Web Authentication . . . 5
2.3 Overview of Web Authentication Mechanisms . . . 7
2.4 The Evolution of WebAuthn . . . 10
2.4.1 Origins: FIDO U2F and UAF . . . 10
2.4.2 WebAuthn Level 1: Standardization the Browser API . . . 11
2.4.3 WebAuthn Level 2: Maturity and Formal Verification . . . 11
2.4.4 The Passkey Initiative . . . 12

3 Technical Background: The FIDO2 Protocol Stack 14
3.1 Architecture Overview . . . 14
3.2 The WebAuthn Specification . . . 15
3.2.1 Registration Ceremony . . . 15
3.2.2 Authentication Ceremony . . . 17
3.3 The CTAP2 Protocol . . . 18
3.4 Attestation Models . . . 20
3.5 Authenticator Types and Credential Models . . . 20
3.6 Passkeys: Synced vs. Device-Bound Credentials . . . 21
3.7 WebAuthn Level 3 Improvements . . . 21
3.7.1 Discoverable Credentials and the Passkey Model . . . 22
3.7.2 The BE and BS Flag Bits . . . 23
3.7.3 Revised signCount Policy . . . 25
3.7.4 The Hybrid Transport Protocol . . . 26
3.8 Cryptographic Algorithm Support . . . 28

4 Design of Passwordless Authentication System 29
4.1 System Requirements Analysis . . . 29
4.1.1 Functional requirements . . . 29
4.1.2 Security requirements . . . 29
4.1.3 Usability requirements . . . 30

4.2 System Architecture Design . . . 30
4.2.1 Presentation Tier . . . 31
4.2.2 Application Tier . . . 31
4.2.3 Data Tier . . . 32
4.3 Credential Lifecycle Design . . . 32
4.4 Security Properties Analysis . . . 33
4.5 Real Life Passkey Authentication Scenario . . . 36

5 Implementation of the Passkey Prototype 37
5.1 Development Methodology . . . 37
5.2 Design Objectives . . . 39
5.3 Technology Stack . . . 39
5.4 System Architecture . . . 40
5.5 Core Data Structures . . . 41
5.6 Challenge Generation . . . 44
5.7 Registration Verification . . . 46
5.8 Authentication Verification . . . 51
5.9 Software Authenticator Emulator . . . 52
5.10 Security Property Test Suite . . . 54

6 Evaluation and Results 56
6.1 Test Suite Design . . . 56
6.2 Test Results and Prototype Validation . . . 58
6.3 Evaluation Criteria Framework . . . 59
6.3.1 Security Criteria . . . 59
6.3.2 Usability Criteria . . . 60
6.3.3 Deployment Criteria . . . 61
6.3.4 Privacy Criteria . . . 62
6.4 Consolidated Evaluation Matrix . . . 63

7 Conclusions 64
7.1 Synthesis of Findings . . . 64
7.2 Applicability to Different Deployment Contexts . . . 65
7.3 Conclusions . . . 65

8 Future Work and Research Directions 67
8.1 Standardization Account Recovery for Passkeys . . . 67
8.2 Platform Providers Trust Concentration and Governance . . . 67
8.3 Post-Quantum Cryptography Integration . . . 67
8.4 Accessibility Engineering for Passkeys . . . 68

9 Glossary of Acronyms 69

Bibliography 70

1 Introduction

The basic mechanism of Internet authentication, **the password**, was designed in the 1960s for shared computer systems and was never intended to protect billions of individual accounts on a hostile global network.

Despite decades of incremental improvements, such as hashing algorithms, salting, multifactor authentication and password managers, the model remains fundamentally defective, because it depends on a shared secret that can be stolen, guessed, phished or leaked on a massive scale.

The **FIDO2 framework**, developed in collaboration between the FIDO Alliance and the W3C, proposes a paradigm shift from authentication using shared secrets to public key authentication and the recent introduction of ***passkeys*** makes this vision viable for the consumer market for the first time.

This chapter establishes the motivation for the research, defines the objectives and research questions and clarifies the scope and limitations of the work.

1.1 Motivation

The arguments in favor of replacing passwords are based primarily on three factors: an increase in security breaches, persistent usability problems and new regulatory requirements.

From a **security perspective**, credential attacks remain the main cause of data breaches.
According to the Verizon's Data Breach Investigations Reports [Ver25], more than 80% of web application hacking incidents are caused by stolen, weak or reused credentials.

This is due to their vulnerability to phishing, users tendency to reuse them across different services, the risks of server breaches and the threats posed by brute force and credential stuffing attacks. All of this is well known and widely documented.

The annual global cost of credential theft is measured in billions of dollars, which includes direct financial losses, regulatory penalties, reputational damage and repair costs.

From a **usability perspective**, passwords impose a significant cognitive overload on users.
The average person manages dozens to hundreds of accounts, making unique, complex passwords impractical without a password manager.

The password reuse across sites means that a single breach can cascade across multiple services (credential stuffing).

The recovery proces, like security questions, email resets or call centres, are all of them expensive for organisations and frustrating for users.

From a **regulatory perspective**, major frameworks are moving towards on phishing resistant authentication.

The NIST SP 800-63-4 [Tem+25] formally classifies passkeys as satisfying Authenticator Assurance Levels (AAL) 2 and 3 depending on the type, as explained in the Section 3.6.

The European Union's eIDAS 2.0 regulation [Eur24] and NIS2 Directive [Eur22] are driving enterprise adoption of FIDO2 across the continent.

This regulatory momentum means that passwordless authentication is not just a security improvement but increasingly becoming a regulatory compliance requirement.

The FIDO framework [All22b] addresses all three requirements simultaneously.

By replacing shared secrets with the asymmetric key pairs linked to the relying party's origin, it eliminates phishing by design.

By using the platform biometrics (fingerprint, face recognition) or hardware security keys for user verification, it removes the need to remember passwords entirely.

And finally by satisfying the highest assurance levels in modern identity frameworks.

1.2 Research Objectives

The main objective of this work is to design, implement and rigorously evaluate a passwordless authentication system based on the WebAuthn standard and the FIDO2 protocol stack in general.

We will pursuit the following five research objectives:

RO1 - Academic Research. Conduct a comprehensive study on web authentication, the relevant literature and its adoption by the industry in order to situate this work within the current research landscape and identify remaining challenges.

RO2 - Technical Analysis. Perform a comprehensive analysis of the FIDO2 protocol stack, focused on the WebAuthn API, the CTAP 2 protocol and the passkey credential model, analysing this standards in depth.

RO3 - System Design. Design a passwordless authentication architecture that supports both synced and device-bound passkeys, integrates conditional UI for seamless user experience and follows relying party best practices published by the FIDO Alliance and major platform providers.

RO4 - Prototype Implementation. Implement a functional prototype based on the designed system that directly complies with the W3C WebAuthn Level 3 specification, without relying on third party WebAuthn libraries, and that demonstrates the full registration, authentication and credential management lifecycle for both synced and device-bound passkeys.

RO5 - Comparative Evaluation. Validate the implemented prototype with an automated security test suite that exercises the registration and authentication ceremonies end-to-end and place the results in context using a comparative matrix that evaluate WebAuthn against convencional web authentication mechanisms.

1.3 Research Questions

The following research questions guide the investigation:

RQ1. What are the security implications of synced passkeys compared to device-bound credentials and how do these differences affect the assurance levels?

RQ2. How does the FIDO2/WebAuthn protocol stack achieve phishing resistant authentication and what specific cryptographic and protocol mechanisms provide this assurance?

RQ3. What design principles and relying party best practices are required to build a robust passwordless authentication architecture that correctly supports WebAuthn, CTAP 2, conditional UI and the full passkey lifecycle?

RQ4. How can a functional prototype be implemented without using third party WebAuthn libraries, while maintaining interoperability across browsers, operating systems and authenticator types?

RQ5. How does the implemented prototype perform under an automated security test suite, and how do its security, usability and deployment complexity characteristics compare with the conventional web authentication mechanisms?

1.4 Scope and Limitations

This research focuses on the web authentication via the JavaScript WebAuthn API and is delivered as a functional prototype implemented directly against the W3C WebAuthn Level 3 specification, no third party WebAuthn library (such as `py-webauthn` [Lab25]).

The prototype supports the ES256 and EdDSA signature algorithms and the `none` and `packed` (self-attestation) attestation formats, runs as a single relying party with an in-memory data tier, and treats the browser, the platform authenticators and the underlying TLS channel as trusted components, that we explain in the trust zones described in Section 4.4.

There are several scenarios that are deliberately out of scope. The federated identity flows involving multiple relying parties or external identity providers (e.g. OpenID Connect) are not addressed because the prototype is single relying party.

The `RS256` algorithm, that Level 3 classifies as a legacy option, and the `tpm`, `apple`, `android-key` and `fido-u2f` attestation formats are refused, to keep the implementation auditable against the specification.

No account recovery endpoint is implemented, because the synchronised passkeys are recovered natively through the platform account and cryptographic recovery protocols are deferred to future research in Section 8.1.

The evaluation in Chapter 6 exercises the server side ceremony logic end-to-end via the Flask test client against the software authenticator emulator (Section 5.9) and the usability ratings of Section 6.3 are derived from the comparative authentication literature rather than from a dedicated user study.

The post-quantum cryptography integration and accessibility engineering for passkeys are out of scope and are treated as future research directions in the Sections 8.3 and 8.4.

2 Academic Research about Web Authentication

In this chapter, we will establish the common theoretical framework that will be used in the rest of the work.

2.1 Prior Literature

The existing literature on web authentication covers security analysis, usability evaluation and cross device authentication.

Each of these aspects offers different perspectives that we must take into account.

Below, we will detail the literature consulted in preparation of this work.

The comparative evaluation matrix performed in this study, as you can see in the Section 6.4, is based on the structure and methodology of the foundational comparative study on web authentication conducted by Bonneau et al. [Bon+12], which evaluated 35 authentication schemes across usability, deployability and security dimensions.

The key finding that passwords offers the lowest level of security while maintaining the highest deployability has proved to be valid, and the resulting tension explains the adoption challenges that more secure mechanisms encounter.

The large scale study done by Lyastani et al.'s [Lya+20] identified account recovery, ease of use across different devices and the formation of mental models as the main barriers to adoption, independent of the technical level of the users.

The field study done by Farke et al. [Far+20] revealed that, although WebAuthn offers strong security features, users often continue to use passwords as a fallback, creating a dual credential attack surface that partially compromises the security benefits of WebAuthn.

The analysis by Owens et al. [OWU21] examined the use of smartphones as FIDO2 roaming authenticators via the hybrid transport protocol, identifying QR code data exchange as a new friction point that requires deliberate user experience design.

Similarly, the research by Würsching et al. [Wür+23] further demonstrated, in the laboratory study to directly compare platform based and roaming authenticators on smartphones, that rare events, such as device loss and account recovery caused the most serious usability failures, confirming that the device binding model was the architecture primary practical weakness.

They also emphasized that the FIDO Alliance should aim for an open neutral provider solution rather than one tied to a single platform's infrastructure.

2.2 Taxonomy of Web Authentication

If we wants to do a rigorous analysis of the security of any authentication mechanism, we need a common conceptual framework that allows us to better understand the subject.

Authentication Factors

The authentication mechanisms are conventionally classified according to the classical three factor model:

- **Knowledge factors**: something the user knows, such as a password, PIN or the answer to a security question.
- **Possession factors**: something the user has, such as a hardware security key or mobile device.
- **Inherence factors**: something the user is, such as a fingerprint, facial geometry or voice pattern.

The security argument for combining factors is that an attacker must simultaneously compromise at least two different factors to succeed.

This argument is only valid when the factors are *independent*, meaning that a single attack vector cannot compromise multiple factors in the same transaction.

Credential Binding Models

Beyond the factor classification, web authentication mechanisms can be distinguished by the binding model that links the credential to a specific server or origin.

A **channel-agnostic** credential has the same value regardless of where it is sent, a captured copy is indistinguishable from one presented legitimately.

Passwords and OTP codes are channel-agnostic, their value don't encode the identity of the intended recipient.

A **channel-bound** credential incorporates cryptographic material specific to the legitimate server's identity into each assertion, making a credential captured from or for a different origin mathematically useless.

The WebAuthn achieves channel binding by including the origin in the signed `clientDataJSON` structure, as detailed in the Chapter 3.

Authenticator Assurance Levels

The document NIST SP 800-63B-4 [Tem+25] formalises the relationship between the factor selection and security assurance into three **Authenticator Assurance Levels** (AALs), which serve as the normative reference in the rest of this work.

Requirement	AAL1	AAL2	AAL3
Permitted Authenticator Types	* *Any AAL2 or AAL3 authenticator type* * Password * Look-up secret * Out-of-band * SF OTP * SF cryptographic	* MF cryptographic * MF out-of-band * MF OTP * Password or biometric comparison plus: –SF cryptographic –Look-up secret –Out-of-band –SF OTP	* MF cryptographic * SF cryptographic plus: –Password –Biometric comparison
FIPS 140 Validation (Government Verifiers and Authenticators)	**Verifiers** –Level 1	**Verifiers** –Level 1 **Authenticators** –Level 1 overall	**Verifiers** –Level 1 **Authenticators** –Level 1 overall
Reauthentication (recommended)	30 days overall	24 hours overall 1 hour inactivity Single factor required	12 hours overall 15 minutes inactivity
Phishing Resistance	Not required	Recommended; Must be available	Required
Replay Resistance	Not required	Required	Required
Authentication Intent	Not required	Recommended	Required
Key Exportability	Permitted	Permitted	Prohibited

Figure 2.1: Authentication Assurance Levels (AAL) requirements as defined in NIST SP 800-63B-4.

Source: NIST Special Publication 800-63B-4 [Tem+25].

- **AAL1** permits single authentication factor of any kind and provides a basic level of assurance suitable for low risk applications.
- **AAL2** requires two factors of different classes, at least one of which must be a cryptographic authenticator protected by approved hardware or software. This level is suitable for most consumer facing web applications that handle personal or financial data.
- **AAL3** is the highest level, requires a hardware bound cryptographic authenticator that provides resistance to verifier impersonation, that ensures the authentication exchange is cryptographically bound to the identity of the legitimate verifier and cannot be replicated by a man-in-the-middle attack or a phishing site.

Credential Portability

Another important concept is the **credential portability**, that is the ability of a user to use the same authentication credential across multiple devices, platforms or contexts without reenrolling it each time.

The device bound credentials are generated inside a hardware authenticator and are never

exported, they are non portable by design, providing the highest level of hardware security at the expense of account recovery complexity.

Then there are two aproachs of archive the credenciatl portablity.

The early approach are the **roaming atuhenticators** that provides portability via hardware that move between devices, like hardware security keys explained in the Section 2.3.

The modern approach are the **synchronised credentials**, like *passkeys* explained in the Section 2.3, that are securely synchronized across devices via cloud platform ecosystems (like Apple iCloud Keychain [App26a] or Google Password Manager [Bir22]) and they are fully portable within a given cloud platform ecosystem at the cost of introducing an additional trust dependency.
The introduction of cloud based key stores managed by platform providers as credential custodians creates attack surfaces that must be taken into account.

The Table 2.1 summarises the four taxonomic dimensions of the mechanisms evaluated in this Chapter, more specifically in the Section 2.3.

Mechanism	Factor class	AAL	Binding model	Portability
Password	Knowledge	1	Channel-agnostic	Full
TOTP / HOTP	Possession	1–2	Channel-agnostic	Seed-portable
Federated (OIDC)	Delegated	1–2	IdP-bound	Full
MFA (pwd + TOTP)	Knowledge + Possession	2	Channel-agnostic	Full
WebAuthn Level 2 (HW)	Possession + Inherence	2–3	Channel-bound	None
WebAuthn Level 3 (passkey)	Possession + Inherence	2	Channel-bound	Platform-scoped

Table 2.1: Taxonomic classification of web authentication mechanisms across four dimensions. AAL values follow NIST SP 800-63B-4 [Tem+25].

2.3 Overview of Web Authentication Mechanisms

The following sections present an overview of the principal web authentication mechanisms that we are using nowadays.

Password Based Authentication

The password authentication has been the universal default mechanism on the web since the earliest multi user network systems.

its main practical advantage is that don't imposes any requirements on the client, as it requires only a keyboard and standards compliant browser.
Also is compatible with all platforms, operating systems and network configurations currently in use.

However, this universality is inseparable from its fundamental security weakness.
A password is a *channel-agnostic* secret, as we seen in the Section 2.2, then no server side countermeasure resolves this structural property.

Best practices on the server side require the storage of a salted hash generated by a memory intensive algorithm.

This makes offline brute-force attacks computationally expensive following a credential store breach, but it does not prevent the breach itself, does not address real time phishing and does not mitigate the reuse of credentials across sites.

Das et al. [Das+18] estimate that the half of users employ the same password across multiple services, creating a structural amplification of any individual breach, because a single compromised credential database exposes accounts on all other services where the same password has been registered.

One-Time Password Mechanisms

The **HOTP** [MRa+05] and the **TOTP** [MRa+11] specifications define deterministic algorithms for generating short-lived numeric codes from a shared secret.

The HOTP computes a HMAC-SHA1 counter value that increases each time a new code is generated and TOTP replaces the counter with the current Unix timestamp divided by a fixed time interval, typically 30 s.

Because each code is derived from a value that changes with every use or at each time interval, a captured code cannot be reused once its validity window expires, providing genuine resistance to replay attacks within that window.

The fundamental limitation of both mechanisms is the absence of origin binding. In other words, both are *channel-agnostic*.

An attacker who operates a fake website and forwards the user's TOTP code to the legitimate server in real time, (a type of attack that can be easily implemented using existing man-in-the-middle frameworks) succeeds even in the presence of a one-time code.

Additionally, both HOTP and TOTP require the server to store the shared seed, reproducing the server breach risk of password storage. This means that a breach of the OTP seed database gives the attacker the ability to generate valid codes for all registered users indefinitely.

Federated Login: OAuth 2.0 and OpenID Connect

The **OAuth 2.0** authorisation framework [Har12] and **OpenID Connect** identity layer [Sak+23] (OIDC) are technologies that allow users to log in to one service using an account from another trusted service, called an **Identity Provider** (IdP), such as a social or enterprise account.

In this model, the user authenticates exclusively with the IdP, while the Relying Party (RP) does not directly handle or store user credentials.
Instead of this, the RP receives a signed identity token (ID token), typically a JSON Web Token (JWT), which asserts the user's identity.

The primary security advantage of this approach is that the RP does not store passwords. Consequently, if the RP is compromised, no authentication credentials are exposed, significantly reducing the risk of credential theft.

However, this model introduces a concentration of trust in the IdP, which becomes a critical component of the system. If the IdP is compromised, all services relying on it may be affected, making it a high value target for attackers.

In summary, while federated login improves usability and reduces credential exposure, its overall security depends on the robustness of the identity provider (IdP).

Additionally, federated login raises privacy concerns. Since the IdP is involved in authentication across multiple services, it can potentially observe and correlate user activity, leading to the creation of cross-service behavioral profiles.

This may conflict with data protection regulations such as the General Data Protection Regulation (GDPR) [PE16], particularly with respect to the principle of data minimisation.

Multi-Factor Authentication

The Multi Factor Authentication (MFA) is a security method that requires users to provide at least two different types of evidence (factors) to prove their identity.

The idea is that even if one factor is compromised, an attacker should not be able to authenticate successfully.

Common examples are combining a password with a time-based one-time password (TOTP) from a mobile application or using a password together with a push notification or a hardware key.

This approach improves security, but its effectiveness depends on how independent the factors are.

For example, combining a password with a TOTP code can still be vulnerable to real-time phishing attacks, where an attacker tricks the user into providing both values and immediately reuses them to log in.

Similarly, push-notification MFA can be exploited through repeated login requests, causing the user to accidentally approve one out of frustration or confusion.

These weaknesses are not necessarily due to poor implementation, but rather to the way these authentication methods are designed.

In summary, while MFA significantly enhances security, it does not completely eliminate certain types of attacks.

Hardware Security Keys

The hardware security keys (e.g., YubiKey or Google Titan) represent the most established form of hardware-bound FIDO2 credentials.

They provide highly secure, phishing resistant authentication by binding the private key to a tamper resistant hardware device that is physically in the user possession.

They meet NIST AAL3 requirements when configured with mandatory user verification [Tem+25].

The main limitations of hardware keys are the requirement for physical possession, the need for a specific registration process and the complexity of account recovery when a key is lost without a registered backup.

These factors have historically limited the adoption of hardware keys, for that only is used in high security enterprise or government environments, even though their security properties are universally superior to software based alternatives.

Passwordless Authentication

In this authentication mechanism each user credential is based on a pair of cryptographic keys, a public key and a private key.

The private key is securely stored inside the user's device (*authenticator*) and assume that never leaves it, while the public key is shared with the application.

During authentication, the device uses the private key to sign a challenge generated by the server, proving the user identity without transmitting any secret information.

Additionally, this process is linked to the specific website requesting authentication, which prevents phishing attacks because credentials cannot be reused on fake websites.

This kind of mechanism are *channel-bound*.

Recently, this type of authentication has evolved into a concept called *passkeys*, as explained in the Section 2.4.4.

The **passkeys** are credentials that can be securely synchronized across a user devices using cloud infrastructure provided by platform vendors. This improves usability by allowing users to log in seamlessly from multiple devices, while maintaining strong security guarantees.

The credential material is only decrypted on registered devices following successful authentication on the cloud platform.

This architecture is designed to preserve the fundamental security properties like phishing resistance, delete the server side shared secrets and origin based key isolation, which we will discuss in detail in Chapter 4.

2.4 The Evolution of WebAuthn

In the following sections we will explain in the beginnings and evolution of the FIDO Alliance and the WebAuth standard.

2.4.1 Origins: FIDO U2F and UAF

The FIDO Alliance was founded in 2012 with the explicit goal of developing open, interoperable authentication standards capable of replacing passwords at Internet scale.

It was the first generation of specifications that was published in 2014, addressed two distinct deployment scenarios:

- **The Universal Authentication Framework (UAF)** was designed to enable passwordless login on mobile devices, enabling biometric verification through a key pair managed by the platform without any shared secrets on the server.
- **The Universal 2nd Factor (U2F)** standard adopted a more conservative deployment model, in which a physical device served as a second factor alongside an existing password, and required pressing a physical button to generate a specific response to a website request.

The U2F was widely adopted as a second factor by Google, GitHub and several financial institutions and its cryptographic model directly influenced the WebAuthn architecture.

Its principal limitation was browser integration, becasue U2F required a browser extension or a proprietary native interface, and there was no standardised JavaScript API accessible to arbitrary web applications.

This integration gap prevented relying parties from deploying U2F without demanding non trivial client side configuration from their users.

2.4.2 WebAuthn Level 1: Standardization the Browser API

The WebAuthn Level 1 [W3C19] was published as a W3C Recommendation in March 2019, resolving the integration gap by embedding the authentication API directly into the browser platform under the `navigator.credentials` namespace.

This made strong public key authentication available to any web application through a uniform JavaScript interface, without extensions, plugins or specific vendor integrations.

All major browser engines incorporated implementations conforming to the Recommendation within a few months of its publication.

The Level 1 established the foundational protocol architecture, which has remained stable across all subsequent versions:

- the three party model of **Relying Party, Client and Authenticator**.
- the **registration ceremony** producing a public key bound to an RP ID.
- the **authentication ceremony** producing a signed assertion over a server-issued challenge and the verified page origin.
- the `clientDataJSON` origin binding mechanism.
- the binary structure `authenticatorData`, which conains the RP ID hash, flag byte and signature counter.
- the COSE encoding [JMP22] [Sch17] for public keys.

2.4.3 WebAuthn Level 2: Maturity and Formal Verification

The WebAuthn Level 2 [W3C21], published as a W3C Recommendation in April 2021, consolidated the experience accumulated during the Level 1 deployment period.

Its principal technical additions were:

- enhanced attestation statement processing,
- normative support for discoverable credentials (also called *resident keys*),
- an expanded set of supported COSE algorithms including EdDSA over Ed25519,
- clarified semantics for cross-platform roaming authenticators
- and more precise definitions of the `signCount` anomaly detection logic.

However, this technical maturity came with a drawback that proved to be the decisive obstacle to widespread consumer adoption: credentials were irrevocably bound to the device on which they were created.

A user who lost their hardware security key, wiped their device or switched platforms faced a recovery situation for which most relying parties had no automated solution.

Multiple independent usability studies identified this limitation as the most significant barrier to FIDO2 adoption among non technical users [Lya+20; Far+20; OWU21; Wür+23], as we discussed in Section 2.1.

2.4.4 The Passkey Initiative

It all began in 2022 when Apple, Google and Microsoft made a coordinated joint announcement of platform level support for multi device FIDO credentials, accompanied by the **FIDO Alliance white paper** [All22b] on the technical design of synchronised credentials.

The announcement was strategically significant for two reasons.

First, simultaneous commitment from the three major platform providers ensured that the resulting standard would have immediate and nearly universal device coverage.

Second, the announcement reframed the narrative around FIDO2, shifting from viewing it as a technology intended for security specialists to presenting it as a widely available feature for consumers.

Since 2022, Apple, Google and Microsoft have been implementing a passkey synchronisation infrastructure within their respective cloud platform ecosystems.

The Apple's implementation, based on iCloud Keychain [Inc23], synchronises passkeys across all devices registered with the same Apple ID using end-to-end encryption.

The Google Password Manager offers equivalent functionality on Android and Chrome across all platforms [Bir22].

The Microsoft's implementation integrates with Windows Hello and the Microsoft Authenticator app [Cor23].

By 2023, Google reported that over one billion users had activated passkeys through the Google Account system [Bir22] and browser and platform support had reached effectively universal coverage across modern consumer devices.

The transition from experimental standard to mainstream deployment occurred with unusual rapidity relative to previous web authentication standards, reflecting both the strength of the vendor commitment and the severity of the usability gap that passkeys addressed.

By 2024, major consumer services, including Google, Apple, GitHub, Microsoft, PayPal and many others, have implemented passkey authentication, marking the transition from a specialised security feature to a widespread credential mechanism.

The developer resource site *passkeys.dev* [All23] documents the growing ecosystem of compatible services and platform libraries.

The browsers support for the WebAuthn API is virtually universal amongst modern browsers [Dev26] nowadays, removing client side compatibility as a barrier to implementation.

Then, nowadays the current state of the art presents two distinct models for implementing WebAuthn:

- **the hardware-bound model**, which offers the strongest security guarantees at the cost of significantly reduced usability.
- **the platform passkey model**, which approximates the security guarantees of hardware-bound credentials, while providing usability comparable to federated login.

The security and usability implications of these models are one of the central topics of this work.

Below is a graphical summary of the entire evolution of WebAuthn as explained in this section.

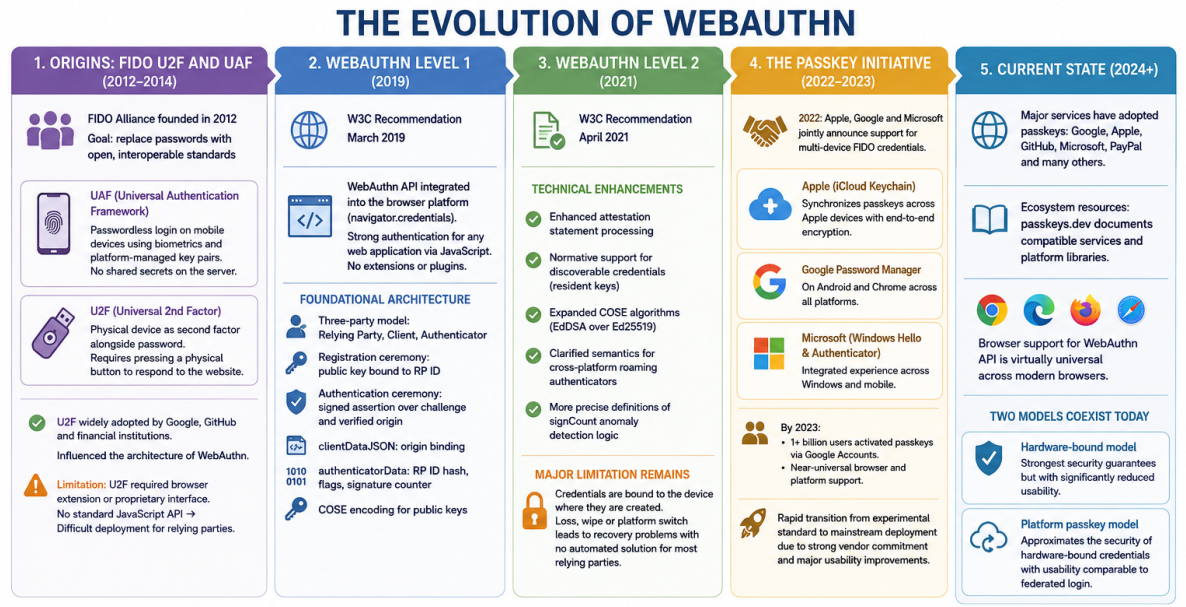


Figure 2.2: An overview of the Evolution of WebAuthn.

3 Technical Background: The FIDO2 Protocol Stack

This chapter provides a detailed technical overview of the FIDO2 protocol stack, covering architectural features, the WebAuthn and CTAP specifications, the cryptographic ceremonies, attestation models, authenticator types and the relevant additions introduced in WebAuthn Level 3 [W3C26] and CTAP 2.2 [All25].

Understanding these mechanisms in depth is essential for the design decisions, implementation choices and security analysis presented in subsequents chapters.

3.1 Architecture Overview

The FIDO2 is an open authentication standard that enables passwordless and multifactor authentication, developed by the FIDO Alliance in collaboration with the W3C.

The FIDO2 protocol stack consists of two main components:

- The **WebAuthn API** [W3C21; W3C26], which defines the browser interface for creating and using public-key credentials.
- The **Client-to-Authenticator Protocol** (CTAP2) [All22a; All25], which specifies the communication channel between the browser platform and external authenticators.

This clear separation of responsabilities enables interoperability because any CTAP compliant authenticator works with any WebAuthn compliant browser without either knowing the other's implementation details.

The FIDO2 architecture is a three party protocol involving the **Relying Party (RP)**, the **Client** (typically a web browser) and the **Authenticator**.

The **Relying Party** is the web application that wants to authenticate users. It generates cryptographic challenges, specifies acceptable credential parameters and verifies authentication assertions.

The **Client** mediates between the relying party (RP) and authenticator, enforcing critical security policies, and most importantly, it constructs the `clientDataJSON` object that binds the ceremony to the page's actual origin, providing the foundation for phishing resistance.

The **Authenticator** generates, stores and operates on asymmetric key pairs, performing user verification through biometrics (e.g., fingerprint), a PIN or simple user presence (e.g., button press).

The Figure 3.1 illustrates the high level architecture and the communication protocols between each component.

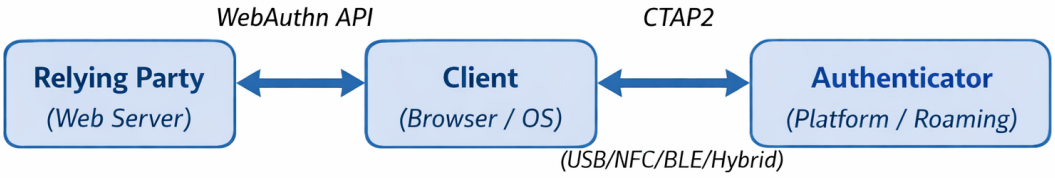


Figure 3.1: High-level FIDO2 architecture showing the separation between the WebAuthn application-layer protocol and the CTAP transport-layer protocol.

3.2 The WebAuthn Specification

The WebAuthn defines two core ceremonies, the **registration** (credential creation) and the **authentication** (assertion generation), each of which involve a structured exchange of cryptographic data between the Relying Party, the Client and the Authenticator.

3.2.1 Registration Ceremony

The **registration ceremony** begins when the relying party (RP) server generates a `PublicKeyCredentialCreationOptions` object.

This object contains several critical fields, such as the following:

- The `challenge` is a cryptographically random byte sequence (minimum 16 bytes, but 32 or more are recommended) that ensures freshness and prevents replay attacks.
- The `rp` field specifies the relying party identifier (an effective domain), which will be cryptographically bound to the credential.
- The `user` field provides a user handle, which is an opaque byte sequence that must not contain personally identifiable information, used by the authenticator for credential selection.
- The `pubKeyCredParams` field is an ordered list of acceptable public-key algorithm identifiers in COSE format. The preferred algorithm is ES256 (ECDSA over the NIST P-256 curve, COSE identifier -7) but EdDSA with Ed25519 (COSE -8) its growing in popularity, as we will see in the Section 3.8.

The browser invokes `navigator.credentials.create()` with these options, forwards the request to an available authenticator via CTAP2 and the authenticator performs the following operations:

- it verifies User Presence (UP) and optionally User Verified (UV),
- generates a new asymmetric key pair,
- and constructs an `attestationObject`.

The `attestationObject` contains the attestation format identifier (`fmt`), the *authenticator data* (`authData`) and the attestation statement (`attStmt`), as shown in the Figure 3.2.

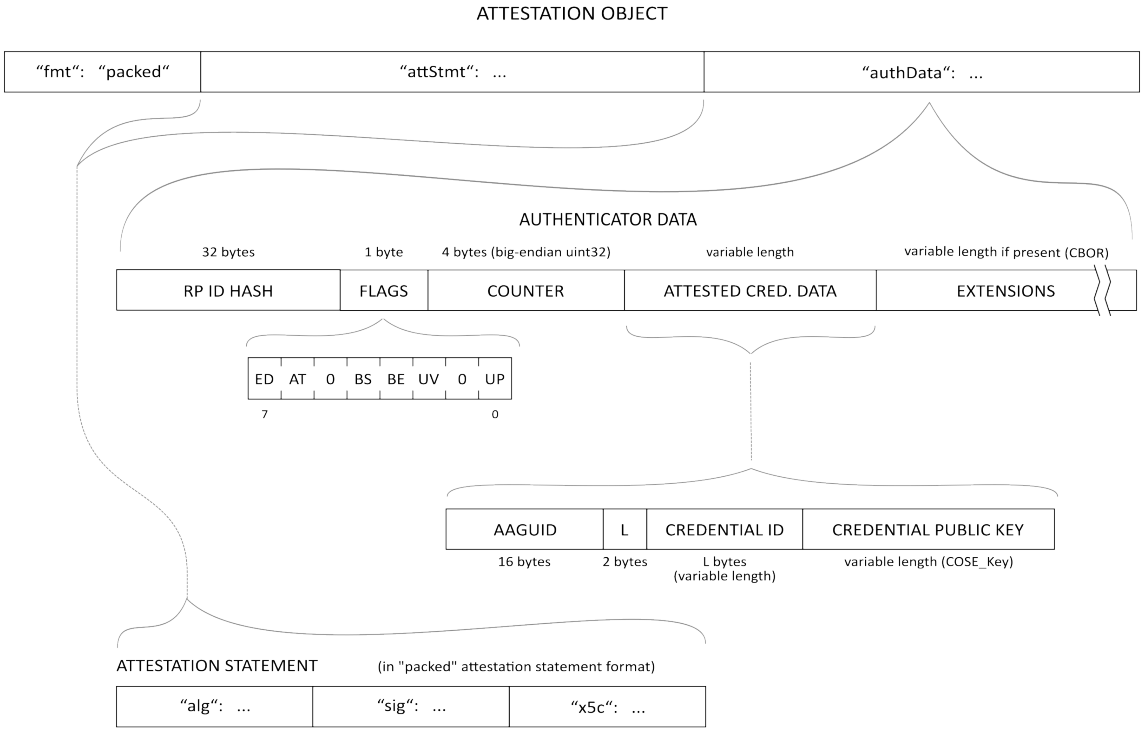


Figure 3.2: Visual representation of the fido attestation structures in WebAuthn L3. *Source:* `https://www.w3.org/TR/webauthn-3/images/fido-attestation-structures.svg`

The `authData` is a Binary Large Object (blob) with a specific sctructure that contains:

- the SHA-256 hash of the rpId (32 bytes).
- a flags byte encoding: Extension Data present (ED), Attested Credential Data present (AT), Backup State (BS) and Backup Eligible (BE) indicators, User Verified (UV) and User Present (UP).
- a 32-bit signature counter.
- if the AT flag is set, the attested credential data comprising the AAGUID (16 byte authenticator model identifier), the credential ID and the COSE encoded public key.

Finally, to represent everything that happens during the registration ceremony, the official WebAuthn API documentation includes a diagram illustrating this process, as shown in Figure 3.3.

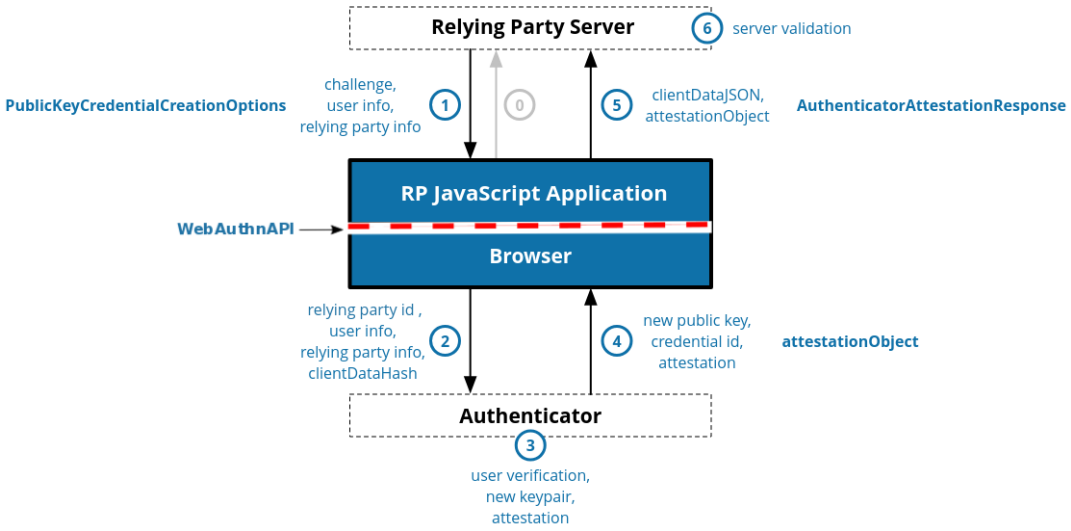


Figure 3.3: Visual flow of the registration ceremony in WebAuthn. *Source:* `https://w3c.github.io/webauthn/images/webauthn-registration-flow-01.svg`

3.2.2 Authentication Ceremony

The **authentication ceremony** follows a symmetric flow respect the registration ceremony, as we can see when comparing the Figures 3.3 and 3.4 .

The relaying party (RP) generates a `PublicKeyCredentialRequestOptions` object that contain a new challenge and, optionally, an `allowCredentials` list specifying which credential IDs are acceptable.

For discoverable credentials (passkeys), the `allowCredentials` list can be omitted entirely, allowing for authentication without a username.

The browser calls `navigator.credentials.get()`, then the authenticator locates a credential matching for the rpId and performs:

- the user verification,
- increments the signature counter,
- generate a digital signature on the concatenation of the authenticator data,
- and create the SHA-256 hash of `clientDataJSON`.

The relying party (RP) verifies the assertion by checking:

- the origin included in `clientDataJSON` matches the expected value,
- the rpIdHash in the authenticator data corresponds to the configured rpId,
- the User Present (UP) and User Verified (UV) flags satisfy the RP's policy,
- the signature is valid respect the stored public key,
- and the signature counter has not decreased, because a decrease indicates potential authenticator cloning.

All of the steps described above can be summarized visually in the following diagram.

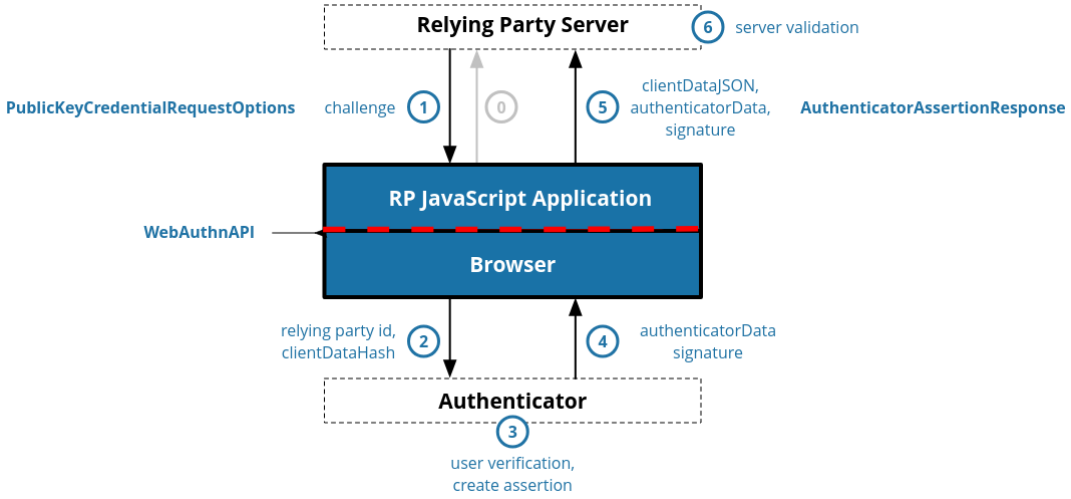


Figure 3.4: Visual flow of the authentication ceremony in WebAuthn. *Source:* `https://w3c.github.io/webauthn/images/webauthn-authentication-flow-01.svg`

3.3 The CTAP2 Protocol

The **Client to Authenticator Protocol** version 2 (CTAP2) is the communication protocol that lets a *client device* such as a browser, operating system or an app talk to an *authenticator* such as a security key, phone or another hardware device.

The CTAP2 is important because it makes passwordless login standardized, secure and interoperable. Without a common protocol, each browser, phone or security key provider would need custom integrations.
For that, the CTAP2 gives them one shared way to communicate.

The CTAP2 operates over four physical transports as we saw in the Figure 3.1, the USB HID, the NFC, the BLE and the newer *hybrid* transport, each with its own specific encapsulation and discovery mechanisms.

The protocol uses CBOR (Concise Binary Object Representation) [Sch17] encoding for all commands and response payloads.

The protocol's core commands map directly to WebAuthn's two ceremonies:

- The `authenticatorMakeCredential` handles registration, creating a new key pair and returning the attestation object.
- The `authenticatorGetAssertion` handles authentication, producing a signed assertion over the challenge and authenticator data.

The Figure 3.5 provides a sequence diagram of both ceremonies with the commands explained above.

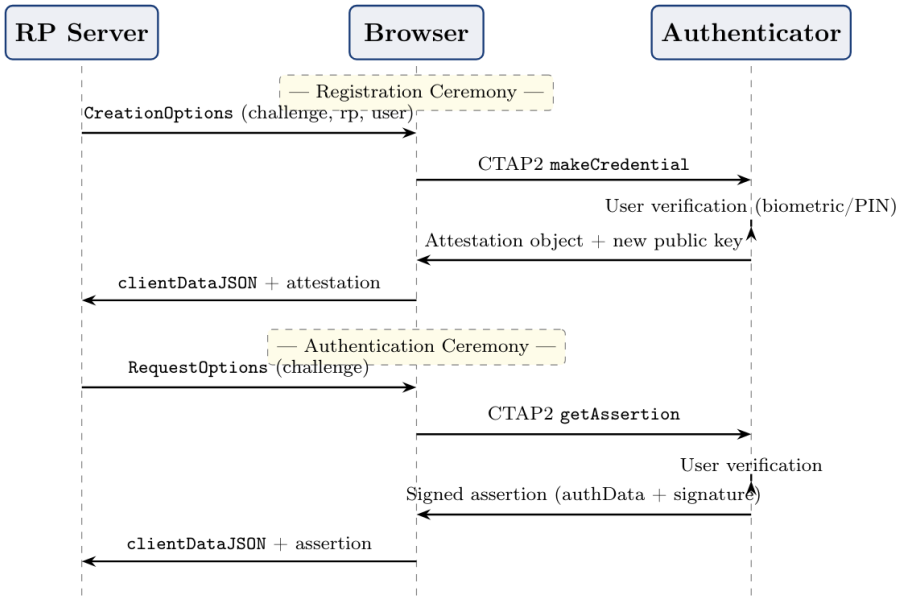


Figure 3.5: Sequence diagram of the WebAuthn registration and authentication ceremonies, showing the three party interaction between the Relying Party server, the browser client and the authenticator.

Also there are two additional commands that are essential for management:

- the `authenticatorGetInfo` allows the client to discover the authenticator's capabilities, supported algorithms and CTAP version.
- the `authenticatorClientPIN` manages the PIN lifecycle.

The Client PIN subprotocol uses Elliptic Curve Diffie-Hellman (ECDH) over P-256 to establish a shared secret between the client and authenticator, which is then used to encrypt the PIN before trasmission.

However, this key agreement is *unauthenticated*, that means the client and authenticator do not prove who they are to each other during the key setup.

The **CTAP 2.2** [All25] introduces support for the *hybrid transport protocol*, enabling cross device authentication where a smartphone serves as a roaming authenticator for a desktop browser session, as we will see later in the Chapter 4.

It also adds enterprise attestation support, credential management commands for authenticator resident credential enumeration and deletion, and the `minPINLength` extension for enforcing minimum PIN complexity.

Also mention that the **CTAP 2.3** [All26] was published as a FIDO Alliance Proposed Standard on February, 2026.

In contrast to the substantial feature additions of CTAP 2.2, version 2.3 is primarily an editorial and clarification release, the FIDO Alliance described it as containing minor and mostly editorial updates, clarifying ambiguities and small corrections.

3.4 Attestation Models

The attestation allows the relying party (RP) to verify the origin and security properties of an authenticator during registration.

The FIDO2 defines several types of attestation, each of which represents a different balance between the assurance of device verification and the user privacy.

- The **Basic (Batch) Attestation** uses a manufacturer key pair shared across a large batch of authenticators (typically 100.000 or more units). This provides proof of the authenticator's model without enabling individual device tracking.
- The **Self Attestation** has the newly created credential key sign its own attestation statement, providing no manufacturer verification but offering maximum user privacy.
- The **Attestation CA (AttCA)** is used primarily with Trusted Platform Modules (TPM), where the authenticator generates unique Attestation Identity Keys signed by a trusted Certificate Authority (CA).
- **None** means that no information is available regarding the origin.

The **FIDO Alliance Metadata Service** (MDS3) provides a centralised repository of authenticator metadata, including attestation root certificates, supported algorithms, security certifications and known vulnerabilities.

The relying parties can query MDS3 to make policy decisions, such as for example a requiring a specific certification level or blocking authenticators with known security issues.

For most consumer facing deployments, requesting no attestation (`attestation: "none"`) is recommended by the FIDO Alliance, as it avoids user consent prompts and privacy concerns.

3.5 Authenticator Types and Credential Models

The authenticators are categorised based on their *attachment modality* (platform versus roaming) and their *credential storage model* (discoverable versus non discoverable).

1. The **Platform authenticators** are integrated into the user's device: Apple's Secure Enclave, Android's hardware backed Keystore, Windows Hello backed by a Trusted Platform Module (TPM) and ChromeOS's built-in authenticator.
2. The **Roaming authenticators** are separate physical devices, such as USB security keys like the YubiKey, NFC enabled tokens like the Google Titan Key or BLE dongles, that connect to the client via external transports.
3. The **Discoverable credentials** (also called *resident keys* in the Level 2 specification) store the user handle and key material directly on the authenticator, enabling the authenticator to enumerate available credentials for a given rpId without the RP needing to provide an `allowCredentials` list. This is the foundation of passkeys and what makes authentication without a username possible.
4. The **Non-discoverable credentials** store only a key identifier (credential ID) on the server side, the authenticator retrieves the private key from this identifier during authentication, so the relaying party (RP) must provide the credential ID in advance.

3.6 Passkeys: Synced vs. Device-Bound Credentials

The term *passkey* was introduced by the FIDO Alliance in March 2022 [All22b] to describe discoverable WebAuthn credentials designed for consumer adoption.

The critical distinction within the passkey ecosystem is between *synced passkeys* and *device-bound passkeys*.

The **Synced passkeys** (multi device credentials) encrypt the private key and synchronise it across devices via cloud credential managers, such as Apple's iCloud Keychain, Google Password Manager, Microsoft's passkey provider or third-party managers like Bitwarden and 1Password.

This solves the device loss recovery problem that had long impeded FIDO adoption, that is if we lose a phone no longer means losing all credentials.
The security trade-off is that the private key is no longer irrevocably bound to a hardware secure element.
Instead of this, its protect by the cloud provider's security infrastructure including end-to-end encryption and account level protections.

The WebAuthn Level 3 [W3C26] exposes the backup state through two new flags in the authenticator data: Backup Eligible (BE) and Backup State (BS), allowing RPs to determine the resilience of credentials at the time of authentication, as we will see later in Section 3.7.2.

On the other hand, the **Device-bound passkeys** maintain the original FIDO2 security model in which the private key never leaves the authenticator's secure element.

The hardware security keys represent the purest form, as they offer the highest guarantee as a possession factor with full attestation capabilities.
But at the cost of reducing usability and widespread adoption.

The NIST SP 800-63-4 [Tem+25] standard, published in 2025, classifies synced passkeys as compliant with Authenticator Assurance Level (AAL) 2 and device-bound passkeys as compilant with AAL3, providing clear regulatory guidance for implementation decisions.

3.7 WebAuthn Level 3 Improvements

The WebAuthn Level 3 [W3C26] is, as of early 2026, the *Candidate Recommendation Snapshot* of the W3C Working Group.

That means is not the recommendation standard yet, but all major consumer services like Google, Apple or Microsoft were implementing since it was already in working draft, as we see above in the Section 2.4.4.

That's why we can say that even if it's not the official standard version, it has already seen widespread adoption and has had time to mature.

To understand what Level 3 adds, it helps to first recall what Level 2 already provides and where it falls short.

In Level 2, a credential is an asymmetric key pair generated inside the authenticator hardware and never exported. The private key lives on exactly one device only.

If that device is lost, stolen or wiped, the credential is gone permanently. This is excellent for security but catastrophic for usability, becasue a user who buys a new phone must manually reenrol every service from scratch.

The Level 3 solves this by allowing credentials to be synchronised across a user's devices through a platform provider cloud infrastructure.

These synchronised credentials are called *passkeys*, as we have already explained in Section 3.6.

Level 3 introduces four concrete technical changes to make passkeys work correctly and securely:

- a **new credential storage model** (discoverable credentials).
- **two new flag bits** in the authentication data structure (BE and BS).
- a **revised counter policy** (signCount value).
- a **new cross-device ceremony** protocol (hybrid transport).

Each is explained in detail below.

3.7.1 Discoverable Credentials and the Passkey Model

A *discoverable credential*, known in the Level 2 specification as a resident key, is a **passkey** that the authenticator stores internally together with all the information needed to use it, including the credential identifier, the relying party identifier and the associated user handle.

The key word is *stored*, because the authenticator can look up and offer the right passkey by itself, without being told in advance which credential to use.

Compare this to the Level 2 non discoverable flow.

Where in that flow, the relying party must send the browser a list of credential identifiers that it considers valid for the user attempting to log in.

In the Level 2 model, discoverable credentials were supported but optional, but in the Level 3 passkey deployment model, they are the primary target.

The practical consequence is a change in the authentication initiation flow.

In a non discoverable credential flow, the relying party must send the browser a list of credential identifiers that it considers valid for the user attempting to log in.

To produce that list, the relying party must first know *who* is trying to log in, which means the user must type their username before the browser even starts the authentication ceremony.

The authentication therefore always required two steps: username first, then cryptographic challenge.

With a discoverable credential the flow collapses into a single step.

The relying party sends an empty `allowCredentials`list.

The authenticator scans its own storage, finds every passkey that belongs to the relying party's domain and presents them to the user in a native selection interface.

The user picks one, the authenticator signs the challenge and the relying party receives a valid assertion that includes the user handle it needs to identify the account.

There is no username field, no first step and no opportunity for the user to mistype their identifier.

How the browser activates this flow. The browser feature that powers username less login is called *conditional mediation*, also referred to as the *autofill* UI.

When a web page loads, the developer calls `navigator.credentials.get()` with the option `mediation: "conditional"` and marks the username input field with the annotation `autocomplete="username webauthn"`.

The browser don't show any pop-up at this point. Instead, it waits silently in the background.

The moment the user taps or clicks the username field, the browser injects the available passkeys directly into the standard autofill dropdown alongside any saved passwords, so the interaction feels identical to selecting a saved username from the browser's password manager.

When the user selects a passkey entry, the device prompts for biometric confirmation and the ceremony completes.

From the developer's perspective, no custom user interface is required.

3.7.2 The BE and BS Flag Bits

Every authentication assertion produced by an authenticator contains a binary structure called `authenticatorData`, as we explained before in Section 3.2.

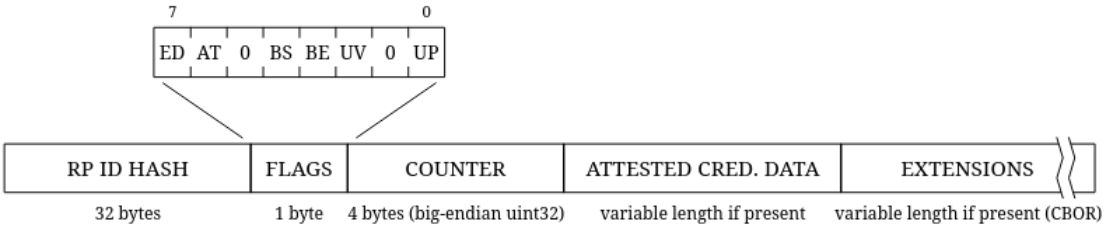


Figure 3.6: Visual representation of the authenticator data structure. Source: [W3C26]

The Byte 32 of this structure is the *flag byte*: a single byte where each bit reports one boolean condition about the assertion.

The Level 2 already defined four bits: UP (User Present), UV (User Verified), AT (Attested Credential Data included) and ED (Extension Data included).

The Level 3 adds two new bits to this byte: BE (Backup Eligible) and BS (Backup State).

They occupy bits 3 and 4 respectively and are both zero in any Level 2 only authenticator, so existing relying party implementations that do not yet inspect them continue to work without change.

The BE flag: Backup Eligible. The BE flag answers the question: *is this credential the type that can be synchronised?*

- The **BE** = 0 means the authenticator that created this credential does not export private keys. The private key exists on exactly one piece of hardware and will never leave it. This is the Level 2 device-bound model.

- The **BE** = 1 means the authenticator does export private keys to a synchronisation service, such as iCloud Keychain or Google Password Manager. The private key may therefore exist on more than one device simultaneously.

Critically, this flag is set at registration and never changes.
A credential does not switch from hardware-bound to synchronised after it is created.

If a Relying Party observes the BE flag changing between a registration and a later authentication for the same credential, it must treat this as an integrity violation and reject the assertion.

Any deployment context that requires hardware assurance (for example, a banking application requiring NIST AAL3 [Tem+25]) must check whether BE = 1 at registration and refuse to register the credential if it is.

The BS flag: Backup State. The BS flag answers the question: *is this specific credential backed up right now?*

- The **BS** = 0 means the synchronisation service don't currently hold a confirmed copy of this credential's private key.
- The **BS** = 1 means the synchronisation service holds a confirmed copy and the credential is available on all of the user's enrolled devices.

Unlike the BE, the BS flag can change between assertions.
A relying party that records the BS value after every authentication has a real time view of the synchronisation state of each credential.

This enables useful operational policies, for example if a relying party could ask a user to check their cloud backup settings if BS drops from 1 to 0, or a security dashboard could alert administrators that certain accounts have credentials that have not been replicated yet.

The four possible combinations. Because the BE (Backup Eligible) and the BS (Backup State) are *independent bits*, four combinations are possible.

The Table 3.1 lists all four and explains what each means in practice.

BE	BS	Meaning and relying party action
0	0	**Hardware-bound, not backed up.** The private key exists on exactly one device and will never leave it. This is the Level 2 device-bound model. Suitable for AAL3.
0	1	**Invalid combination.** A credential that is not backup-eligible cannot logically be in a backed-up state. The Level 3 specification requires the Relying Party to reject any assertion with this combination.
1	0	**Synchronisable, not yet backed up.** The credential is a passkey but has not been yet confirmed by the synchronisation service. The private key may exist only on one device for now. Suitable for AAL2. The Relying Party may wish to monitor for a transition to BS = 1.
1	1	**Synchronisable and backed up.** The synchronisation service holds a confirmed copy. The credential is available on all of the user's enrolled devices. Suitable for AAL2.

Table 3.1: BE/BS flag combinations and their interpretations per WebAuthn Level 3 [W3C26].

3.7.3 Revised signCount Policy

What signCount is and why Level 2 used it. Every `authenticatorData` structure contains a 32-bit integer called the `signCount` as we can see in the Figure 3.6 like COUNTER.

In Level 2, this counter is maintained inside the authenticator and incremented by at least 1 on every authentication assertion.
After each successful authentication, the relying party saves the counter value it just received.

Before accepting any future assertion, it checks that the new counter is strictly greater than the saved one. If it is not, something is wrong.

The purpose of this check is to identify a potential **clone detection**.

If an attacker somehow extracted the private key from a hardware authenticator and installed it on a second device, they would have two devices that can produce valid signatures. But each device maintains its own independent counter.

We will explain an example, if a legitimate user authenticates from their real device (counter goes to 10), and the attacker later authenticates from their clone (counter is at 7), the relying party will receive a value of 7 when it expects something greater than 10.
The discrepancy is a strong signal that a clone is in use.

Why this breaks for passkeys. The synchronised passkeys are, by design, intentional clones.
When a passkey is synchronised to a user's laptop and their phone, both devices hold an identical copy of the private key.

The Level 2 counter policy would fire false positives continuously in normal multi device usage.

The Level 3 solution. The Level 3 resolves this by introducing a simple conditional rule based on the BE flag:

- If **BE** $= 0$ **(hardware-bound credential)**: apply the full Level 2 `signCount` check. Any value not strictly greater than the stored value is an anomaly that must be take into account.
- If **BE** $= 1$ **(synchronised passkey)**: the authenticator should set `signCount` to 0 in every assertion. The relying party must treat `signCount` $= 0$ as normal and must not raise an anomaly error. Counter based clone detection is simply not available for this credential.

The consequence is that synchronised passkeys sacrifice the ability to detect clones in exchange for seamless operation across multiple devices.

This is a *deliberate decision* explicitly included in the specification, not an omission.

The relying party must record the BE value at the time of registration in order to apply the correct policy during each subsequent authentication.

Therefore, a relying party implementation that always applies the Level 2 anomaly rule (regardless of the BE flag) will stop working for all users who log in with a passkey from more than one device.

3.7.4 The Hybrid Transport Protocol

The most architecturally significant addition in CTAP 2.2 [All25] and WebAuthn Level 3 [W3C26] is the **hybrid transport protocol**, which enables a mobile phone to act as a roaming FIDO2 authenticator for a desktop browser on a different device.

This addresses the scenario in which a user's passkeys are stored on a phone but they wish to authenticate to a service accessed via a desktop computer, as we will see in the Section 4.5.

Before Level 3, there was no standardised way to use the phone's authenticator from the desktop browser. The phone could not be treated as a roaming authenticator over any protocol that a browser exposed it.

The hybrid transport defines exactly how to bridge this gap, how the browser on the desktop and the authenticator on the phone communicate through a cloud meditation relay channel, with end-to-end encryption so that neither the relay server nor the network can read the contents.

Step-by-step ceremony walkthrough. The hybrid ceremony proceeds in four sequential phases, illustrated in Figure 3.7.

- **Phase 1 — Proximity Establishment.**

  The desktop browser generates an ephemeral asymmetric key pair and a tunnel server identifier. It encodes both into a QR code and displays it on a screen.

  Then the user scans the QR code with their phone's camera. This scanning step serves a security purpose, it enforces physical proximity, because the phone must be in the same room as the desktop to scan the code.

  A remote attacker cannot trigger this step without physical access to the screen.

- **Phase 2 — Encrypted Channel Establishment.**

  Both the desktop browser and the smartphone connect independently to the tunnel server identified in the QR code.

  Then they perform a *cryptographic handshake* using the Noise protocol [Per18], as specified in CTAP 2.2 [All25].

  This handshake derives a shared session key known only to the two devices.

  From this point forward, every message between the browser and the phone is encrypted with this key.

  The tunnel server forwards the ciphertext but cannot decrypt or modify it.

  If anyone tampers with a message in transit, the decryption will fail and the ceremony is aborted.

- **Phase 3 — Authentication Ceremony.**

  The browser sends the CTAP2 command `authenticatorGetAssertion` through the encrypted channel to the phone, exactly as it would if the phone were a USB connected security key.

The phone receives the command, displays a confirmation prompt, asks the user to verify their identity using biometrics or a PIN, and then generates the signed `authenticatorData` assertion using the passkey stored in its secure enclave.

The signed assertion is sent back through the encrypted channel to the browser.

- **Phase 4 — Completion.**

  The browser decrypts and extracts the assertion, then submits it to the relying party's server endpoint in the standard WebAuthn response format.

  From the relying party's perspective, this response is identical to one produced by a local platform authenticator on the same device.

  The relying party does not need to know or care about if the authenticator was on a different physical device.

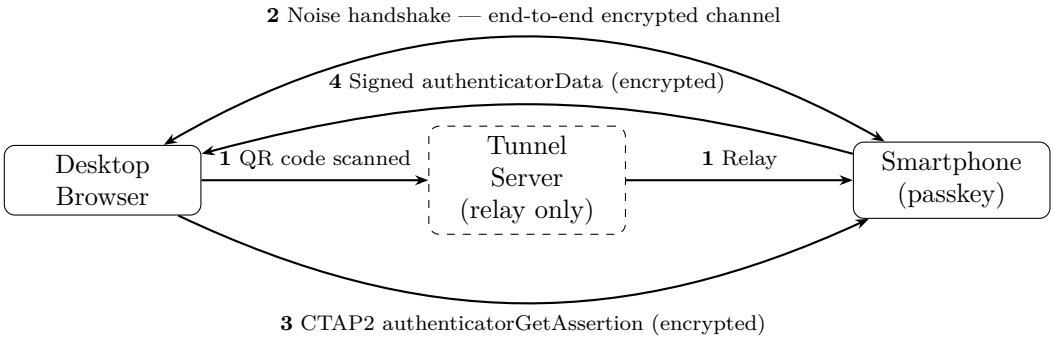


Figure 3.7: The four phases of the hybrid transport ceremony per WebAuthn Level 3 [W3C26]. The tunnel server forwards messages after Phase 2 but cannot read or modify them.

Security properties of the hybrid transport. The hybrid transport provides three security guarantees that are worth mentioning explicitly.

- The **Channel confidentiality.** All CTAP2 messages and authentication assertions are encrypted with the Noise-derived session key. The tunnel server, the network operator and any passive eavesdropper see only ciphertext.

- The **Channel integrity.** The *AESGCM* cipher provides authenticated encryption, that makes that any in transit modification of a message will be detected and the ceremony will abort.

- The **Origin binding is preserved.** The WebAuthn origin binding mechanism works identically in the hybrid flow.

  The browser on the desktop still inserts the legitimate relying party origin into `clientDataJSON`, the phone signs over this value just as it would in a local ceremony and the relying party verifies the origin in the normal way.

  A phishing site cannot take advantage of the hybrid transport because it cannot impersonate the legitimate origin in the signed data.

3.8 Cryptographic Algorithm Support

We will summarize the cryptographic algorithms supported by the WebAuthn standard.

Compared to WebAuthn Level 2, the Level 3 evolution does not introduce a radical change in the underlying cryptographic architecture, but instead expands practical support for modern COSE algorithms.

The **ES256** (COSE -7), **RS256** (COSE -257) and **EdDSA/Ed25519** (COSE -8) are the primary algorithms in WebAuthn.

In particular, EdDSA over Ed25519 (COSE -8) is growing in adoption in many authenticators and passkey ecosystems, because its offering improved performance, compact signature format and implementation simplicity over traditional ECDSA.

But even so, the dominant algorithm in current deployments remains ES256 (ECDSA P-256, COSE -7) due to its widespread compatibility with many hardware and browsers.

In the Table 3.8 presents an overview of the principal supported algorithms.

It should be noted that all currently specified WebAuthn algorithms rely on the hardness of either the **integer factorisation problem RSA** or the **elliptic-curve discrete logarithm problem** (ECDSA, EdDSA).

Both problems are vulnerable to Shor's quantum algorithm [Sho94] and the NIST post-quantum standardisation process [ST25] has selected replacement signature schemes that don't share this vulnerability.

Although this is an open standardisation problem and we will discuss it in future Chapter 8, it's outside the scope of this work.

COSE `alg`	Name	Key type	Security	Notes
−7	ES256	EC P-256	∼128-bit	Most widely deployed
−8	EdDSA	Ed25519	∼128-bit	Fast; compact signatures
−35	ES384	EC P-384	∼192-bit	Higher assurance contexts
−257	RS256	RSA-PKCS1 2048	∼112-bit	Legacy compatibility
−37	PS256	RSA-PSS 2048	∼112-bit	Preferred over RS256

Table 3.2: Principal WebAuthn Level 3 supported algorithms.

4 Design of Passwordless Authentication System

This chapter defines an architectural design for the passwordless authentication system, covering requirements analysis, the high level system architecture, the credential lifecycle and the security properties of the design.

The design decisions are based on the technical characterisation of the FIDO2 protocol presented in Chapter 3,

Also the design serves as the baseline for the implementation of the prototype, as we will see later in Chapter 5.

4.1 System Requirements Analysis

The system requirements are based on the research objectives defined in Section 1.2 and the lessons learned from the prior literature reviewed in the Section 2.1.

The requirements are organised into functional, security and usability categories.

4.1.1 Functional requirements

The system must support the complete WebAuthn Level 3 registration and authentication ceremonies, including the processing of BE/BS flag bits and the revised `signCount` policy for synchronised passkeys.

It must transparently manage both synced and device-bound credentials, implementing discoverable credential support for username less authentication via conditional mediation.

A credential management interface must allow users to register, inspect and revoke passkeys.

During the transition period, backwards compatibility with password-based login must be maintained.

4.1.2 Security requirements

No shared secrets (passwords or OTP seeds) may be stored or transmitted after passkey enrollment.

All WebAuthn operations must use server generated cryptographic challenges with a minimum of 32 bytes of entropy, stored server side with a short time-to-live (60 to 120 seconds) and deleted immediately after use.

The system must validate origin, rpIdHash, user presence and user verification flags according to relying party policy.

The session tokens must be issued with secure attributes (`HttpOnly`, `Secure`, `SameSite`).

The system must enforce the Level 3 `signCount` policy, applying monotonicity checks only to hardware-bound credentials and accepting `signCount = 0` from synchronised passkeys.

At the policy level, the system must be capable of rejecting backup eligible credentials (`BE = 1`) at registration time for contexts requiring hardware-bound guarantees.

4.1.3 Usability requirements

The registration and authentication must complete within measurable time thresholds comparable to or better than password plus MFA flows.

Error messages must guide users toward recovery actions rather than displaying cryptic technical information.

The system must degrade gracefully when WebAuthn is not supported by the browser, falling back to password authentication.

4.2 System Architecture Design

The proposed system follows a classical three-tier web applications architecture adapted to the specific requirements of WebAuthn ceremonies, as shown in Figure 4.1.

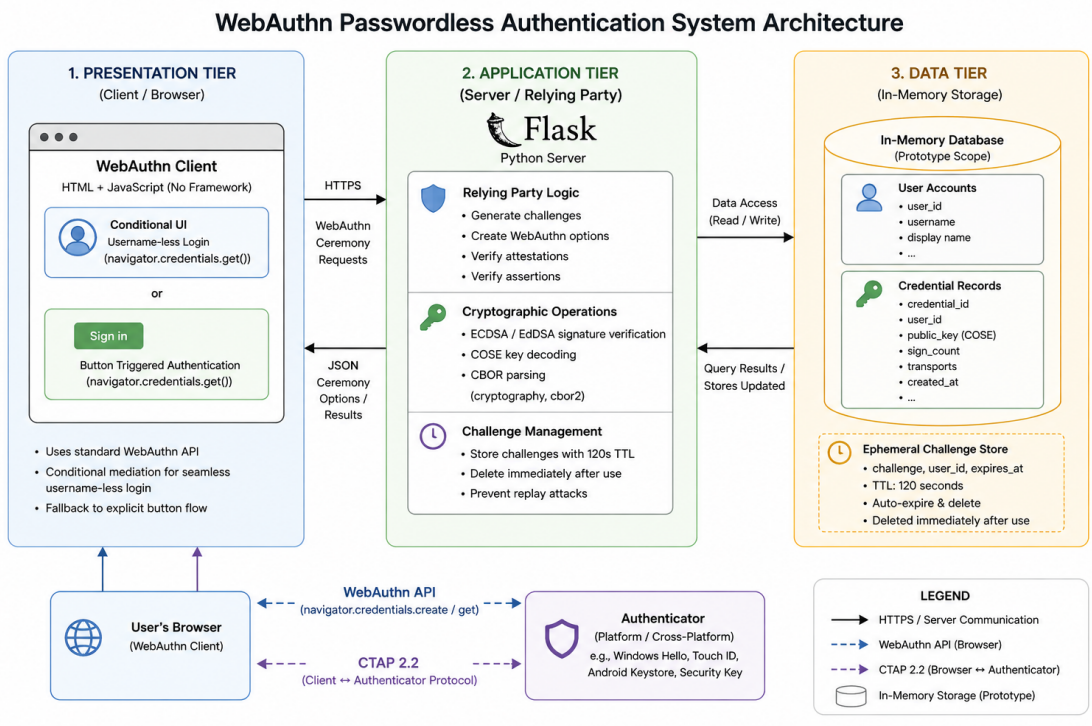


Figure 4.1: Diagram of a passwordeless authentication system architecture.

This separation of responsibilities allows authentication logic to be implemented in a way that facilitates maintenance and testing, while complying with the architectural responsibilities defined by the WebAuthn standard.

In this model, the presentation tier manages user interaction in the browser, the application tier implements the relying party's logic and the data tier stores the account and credential information needed to manage registration and authentication.

4.2.1 Presentation Tier

The **presentation tier** consists of a login HTML page served to the user browser without the use of a JavaScript framework and a complemented by a post login dashboard view shown after a successful authentication.

This design choice is appropriate for a prototype because it minimizes client side complexity, reduces dependency overhead and makes the WebAuthn flow easier to inspect and validate.

The page uses the standard browser WebAuthn API calls functions (`navigator.credentials.create()` and `navigator.credentials.get()`), which are the two primary entry points for registration and authentication ceremonies.

In this tier, the browser acts as the WebAuthn client, mediating communication between the web application and the authenticator.

The implementation supports two complementary interaction modes.

First, it enables **conditional mediation**, which allows the browser to present available passkeys when the user focuses on the login field, thereby supporting a username less flow.

This is particularly relevant for passkey based authentication, since the user may not need to type a username before the browser can discover a suitable credential.

Second, the page provides an explicit button for trigger the authentication flow as a fallback, ensuring that the system remains usable even when conditional UI is unavailable or unsupported by the browser

4.2.2 Application Tier

The **application tier** could be implemented as a Flask based Python server (or another microframework), which acts as the WebAuthn relying party.

This tier is responsible for the protocol logic required to coordinate both registration and authentication ceremonies.

In practice, this means that the server generates cryptographically strong challenges, constructs the ceremony option objects sent to the browser and verifies the responses returned by the authenticator through the browser.

During **registration**, the server creates a `PublicKeyCredentialCreationOptions` object that includes the relying party identifier, user information, challenge, supported cryptographic algorithms and authenticator selection criteria.

During **authentication**, it generates a `PublicKeyCredentialRequestOptions` object that contains a fresh challenge and if are require also do the list of allowed credentials.

In both cases, the server must ensure that the challenge is associated with the correct user and ceremony context so that the later response can be validated correctly, as we explained earlier in the Section 3.2.

The server also performs the essential verification steps defined by the WebAuthn model.

The `cryptography` library is used for signature verification, including ECDSA and EdDSA operations and the `cbor2` library is used to parse CBOR-encoded WebAuthn structures such as the attestation object and COSE public keys.

Finally, it should be mentioned that the Flask server acts as the central control point of the architecture. It serves as the trusted component that validates all credential operations and maintains the authoritative record of users and credentials.
This makes the application tier the most critical component for the system.

4.2.3 Data Tier

The **data tier** stores user accounts, credentials and short-lived authentication challenges.

The credential records include the credential identifier, public key, sign counter and any transport or metadata information that is relevant for later authentication.

These records allow the relying party to match an assertion returned by the browser to the account that originally registered the credential.

A particularly important aspect of the data tier is **challenge management**.
Each challenge generated by the server is stored temporarily with an automatic expiration time, the reasonable interval could be between 60 and 120 seconds. The lower time is preferred to keep the replay window as short as possible without forcing a re-prompt during a normal biometric ceremony.

Once the challenge expires or is used successfully in a ceremony, it's deleted immediately.
This behavior is necessary to preserve the freshness requirement of WebAuthn and to reduce replay attack risk.

Because if challenges are stored indefinitely, an attacker could potentially reuse old values to attempt unauthorized authentication.

If the system is not intended for production deployment, the in memory storage model may be appropriate, since the lack of persistent storage is acceptable and helps isolate the security properties from the authentication flow itself.

However, in a production implementation, the data tier would need to be replaced with a durable storage mechanism, like a persistent database with additional controls for encryption, backup, access control and audit logging.

4.3 Credential Lifecycle Design

In this section we will explain in detail the lifecycle of credentials and what happens in each phase.
The credential lifecycle consists of four phases:

- the **creation** (registration)
- the **usage** (authentication)
- the **management** (inspection and revocation)
- the **recovery** (recover access after device loss)

During the **creation**, the system generates a `PublicKeyCredentialCreationOptions` object configured for passkey best practices.

The `residentKey` parameter is set to `"required"` to ensure discoverable credentials, enabling username less authentication via conditional mediation.

The `userVerification` parameter is set to `"required"`, enforcing biometric or PIN verification.

The `pubKeyCredParams` list orders ES256 (COSE -7) first, with EdDSA (COSE -8) as a second option or optionally also can use RS256 (COSE -257) for legacy compatibility.

At the time of registration, the server records the values of the BE and BS flag from the attestation response and stores the credential with the status of synchronized (`is_synced = True`) or hardware-bound (`is_synced = False`).

During the **usage**, the system supports two authentication paths.

The primary path uses conditional mediation, where a WebAuthn `get()` call with `mediation: "conditional"` is initiated on page load, causing available passkeys to appear in the browser's autofill dropdown.

The secondary path is explicit modal authentication via a dedicated button.

In both cases, the server applies the Level 3 `signCount` policy, as we explained in detail in the Section 3.7.3.

During the **management**, the system allows the relying party to enumerate stored credentials and revoke it that are not longer trusted.

For contexts requiring NIST AAL3 assurance, the system can reject credentials with `BE = 1` at registration time, ensuring that only hardware-bound credentials are accepted.

The **recovery** phase is addressed structurally by the synchronisation model for passkeys, because a user who loses a single device retains access through their remaining enrolled devices.

For hardware-bound credentials, recovery requires registering backup credentials in advance, a limitation that the passkey model is specifically designed to overcome.

4.4 Security Properties Analysis

In this section, we will analyze the security properties that WebAuthn Level 3 can offer with synchronised passkeys, starting by identifying the assets that need to be protected, then dividing the system into trust zones and finally defines a list of security features that should be reviewed.

Assets Under Protection

The following primary assets are relevant in the synchronised passkey architecture.

- **Credential private key material.** The private keys generated during registration is the highest value asset in the system.

  The exposure of a private key to an unauthorised party allows for the falsification of authentication assertions for all relying parties associated with that credential.

- **Platform account credentials.** The authentication credentials (password, recovery key, trusted device token) used to access the platform account (Apple ID, Google Account, Microsoft Account) that governs synchronisation.

  Compromise of these credentials is an indirect path to the credential private key material.

- **Ceremony integrity.** The correctness of the registration and authentication ceremonies as executed by the browser and verified by the RP server.

  Any manipulation of ceremony parameters (challenge, origin, signed data) that is accepted by the server constitutes a breach of ceremony integrity.

- **Device credential store.** The device storage location (Secure Enclave, Keystore or Credential Manager) from which the private key is accessed at authentication time.

  Physical or logical access to this storage on a compromised device may permit key extraction.

Trust Zones Boundaries

The synchronised passkey architecture is decomposed into four trust zones.

The Figure 4.2 illustrates the zones and the data flows between them.

- **Zone 1 — The Relying Party.** The RP server generates challenges, specifies ceremony policy and verifies assertions. its attack surface is limited to the signed assertion and `clientDataJSON` received over HTTPS.

  The RP cannot observe or influence anything within the platform trust boundary.

- **Zone 2 — The Client (Browser).** The browser enforces origin binding by embedding the current origin in `clientDataJSON` before passing it to the authenticator for signing.

  A compromised browser could subvert this, but such an attack would equally defeat any browser-mediated authentication mechanism.

- **Zone 3 — The Device Credential Store.** The platform authenticator generates and stores private key material in a hardware isolated environment (Secure Enclave, TPM or equivalent).

  Attacks that remain outside the hardware boundary cannot reach key material.

- **Zone 4 — The Provider Cloud Infrastructure.** This zone encompasses the synchronisation channel, the cloud keystore and the provider Hardware Security Module (HSM) infrastructure.

  Its the zone introduced by passkey synchronisation that has no equivalent in the hardware-bound model.

The most consequential cross zone data flow is the credential synchronisation path from Zone 3 to Zone 4, because the encrypted passkey material leaves the device's hardware security boundary and enters the provider's cloud infrastructure.

The security of the entire synchronised passkey model depends on the correctness and integrity of this boundary crossing.

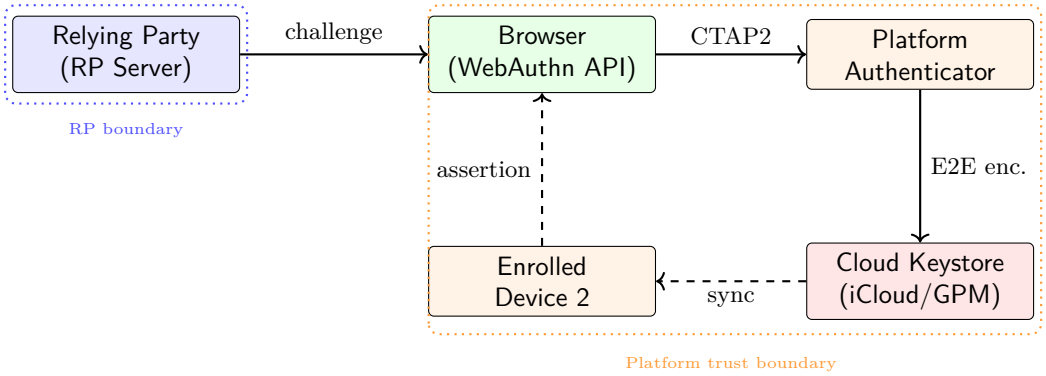


Figure 4.2: Trust boundaries in the synchronised passkey architecture.

Comparison with Hardware-Bound Trust Architecture

In the hardware-bound Level 2 model, the private key is permanently bound to a tamper resistant hardware device.

The Zone 4 does not exist, because there is no cloud infrastructure, no platform account dependency and no synchronisation channel.

The synchronisation model expands the attack surface by introducing the platform account as an indirect authentication gateway.

Since the platform account authentication typically uses the same mechanisms that are open to vulnerabilities, such as email recovery, SMS OTP second factors or passwords that are vulnerable to phishing.

We can observe that the actual security of a synchronised passkey is determined by the security of the platform account, not by WebAuthn's cryptographic architecture.

Level 3 Security Properties Summary

Below is the Table 4.1, which lists the security features that should be checked by design when implementing a secure passwordless authentication method based on WebAuthn Level 3.

More specifically, reflect the architectural changes introduced by synchronised passkeys and the hybrid transport protocol, as we already discussed in the Section 3.7.

Property	Status	Notes
Phishing resistance	✓	Origin binding in `clientDataJSON` preserved
Replay prevention	✓	Single use Challenge
No server shared secret	✓	Server stores public key only
MITM resistance	✓	Origin binding defeats relay attacks
Clone detection (hw key)	✓	Monotonic `signCount`; not available for passkeys
Clone detection (passkey)	×	`signCount` = 0 by design; clone indistinguishable
Per-origin key isolation	✓	Distinct key per $(rpId, user)$
Platform account dependency	New	Sync relies on platform account integrity
Hybrid proximity binding	✓	BLE proximity mitigates remote relay

Table 4.1: Security properties of WebAuthn Level 3 with synchronised passkeys.

4.5 Real Life Passkey Authentication Scenario

A common passkey use case begins when a user creates a passkey on a mobile device, such as an Android phone or an iPhone, and later uses that same passkey to sign in from another device, for example a Windows laptop.

This situation shows one of the main ideas behind WebAuthn Level 3, that is the user can access the same account from different devices without needing to create a new password or register every device separately.

At registration time, the user opens the service on the mobile phone and chooses to create a passkey.
The authenticator generates a key pair, keeps the private key on the device or in the synchronized passkey system and sends the public key to the relying party.

If the credential is designed to be synchronized, the authenticator marks it as backup eligible using the `BE` flag.
Once the passkey has already been synchronized, the `BS` flag indicates that the credential is currently backed up.

The relying party stores the public key, the credential identifier and the associated metadata.
At this point, the account is ready to be used later from other devices.

Later, when the user opens the same service from a Windows laptop that has never been used before, the browser can show a QR code.

The user scans it with the mobile phone where the passkey already exists.
This hybrid flow allows the laptop and the phone to work together so that the phone can act as the authenticator while the laptop acts as the client device, as we saw in more technical detail in the Section 3.7.4.

In some cases, the browser can also use conditional mediation, which makes it possible to sign in without typing a username first.

After the QR code is scanned, the phone asks the user to confirm the login with Face ID, a fingerprint or a PIN.
Once the user approves, the phone creates the authentication response and sends it back through the laptop to the relying party.

The relying party then checks the response, confirms that it belongs to the correct account, and completes the login.
If the credential is synchronized, the `BE` and `BS` flags help indicate that the passkey is a multi device credential and that it's currently backed up.

In this way, the user can sign in from a new device without entering a password, while still using the same passkey created on the phone.

This is the kind of experience that WebAuthn Level 3 is designed to support with a simple login flow that works across devices and feels familiar to the user.

5 Implementation of the Passkey Prototype

5.1 Development Methodology

The prototype was not developed in on the fly or simply by writing code.

It was produced following a **spec-driven development** methodology [Pis26], in which the implementation is at each stage is guided by a written specification rather than by the developer's personal intuition about a certain functions.

This section describes this methodology, both because it has had a significant influence on the resulting code and because it forms part of the contribution.

The methodology is based on four pillars:

- a **formal design** that serves as the specification,
- a set of **structured Markdown** documents that translate that design into applicable rules,
- an **iterative cycle** of code generation,
- and a **manual review** that removes any code that does not directly comply with the specification.

Spec-Driven Development Approach

The Chapter 4 is the formal specification of the prototype.

It defines the system architecture in three tiers, the credential lifecycle across its four phases (creation, usage, management and recovery) and the security properties that the implementation must satisfy, including challenge hygiene, replay prevention, origin binding, the asymmetric `signCount` policy and the handling of the `BE` and `BS` flag bits.

Under this methodology, the implementation is treated as a function of the specification. Every component, every validation step and every data structure in the code is expected to map back to a requirement mentioned in Chapter 4.

Use of Structured Markdown Specification Files

A spec-driven workflow needs a representation of the specification that is operational, that is, close enough to the code to constrain it during development.

To that end, three Markdown documents were produced as **intermediate artefacts** between the academic design of Chapter 4 and the source files of the prototype.

- The `CLAUDE.md` is the main specification document used during development. It restates, in operational form, the system behavior, the layered architecture of the relying party, the registration and authentication flows and the rules that the code must respect (no third-party WebAuthn library, asymmetric `signCount` policy, mandatory ceremony validations, etc.). It functions as a contract between design and code.

- The `PROTOTYPE.md` captures the implementation constraints of the proof of concept. It enumerates the functional, security and usability requirements based from Chapter 4 and fixes the simplifications that are admissible for a PoC, such as the in memory data tier, the restricted algorithm set (ES256 and EdDSA) and the restricted attestation formats (`none` and `packed`). Its role is to enforce a deliberate boundary between what the prototype must demonstrate and what would only belong in a production deployment.

- The `README.md` provides an overview of the prototype, the structure of the repository and the instructions to run the server and the test suite. It is the entry point for any reader, including the academic evaluator and ensures that the prototype is reproducible from the documentation alone.

These three documents act as a controlled bridge between the academic design and the source code.

By making the specification explicit at the level at which code is written, they ensure that the implementation remains reproducible, reviewable and aligned with Chapter 4 throughout the development cycle.

Code Generation and Refinement Process

The source code of the prototype was produced **iteratively** rather than in a single pass.

At each iteration, the relevant portion of the specification, namely a section of Chapter 4 together with the corresponding clauses of the Markdown documents, was used as the input from which a candidate implementation was generated.

The candidate was then evaluated against three references simultaneously: the design described in Chapter 4, the operational rules expressed in the Markdown specification files and the security and functional requirements derived from them.

Discrepancies between the candidate code and any of these three references were treated as defects of the implementation and resolved by revising the code in the next iteration, never by silently relaxing the specification.

Where the design and the Markdown documents disagreed, the disagreement itself was surfaced and resolved explicitly before the iteration could close.

This iterative loop produced an implementation in which each module, file and function corresponds to an identifiable element of the specification.

Manual Review and Simplification

The final stage of the methodology is a deliberate manual review of all generated code.

Every module of the prototype was inspected against the specification, exercised through the automated test suite (Section 6.1) and then **simplified** where the code went beyond what the specification required.

The criteria of this review were, in order, correctness with respect to the specification, readability for an academic reader and minimality of the implementation.

Code that don't contribute to a specified behavior, such as unused abstractions, defensive checks for impossible inputs and over engineered helpers, was removed.

The outcome is an implementation that retains only the functionality strictly required for the proof of concept and that prioritizes **clarity**, **traceability** and **ease of understanding** over performance optimisations or feature richness.

5.2 Design Objectives

The prototype implements a login web application using passkeys as athentication mechanism.

Its implementation follows **six objectives** in alignment with the research goals.

- First, the **full server side registration and authentication verification logic** is implemented directly against the W3C WebAuthn Level 3 specification [W3C26].

  No third party WebAuthn library is used for ceremony orchestration, we only use the libaries `cryptography` and `cbor2`.

- Second, the two algorithms most widely deployed in platform authenticators are used, that was the **ES256** (ECDSA over NIST P-256, COSE `alg=-7`) and the **EdDSA** (Ed25519, COSE `alg=-8`). Any other algorithm is rejected at registration.

- Third, **attestation statement parsing** is implemented for the `none` and `packed` formats (self-attestation variant only), the other formats are refused to simplify the prototype.

- Fourth, the **prototype validates every structural field** that the specification requires on both ceremonies: the single use challenge, the `origin`, the `rpIdHash`, the UP and UV flags, the `clientData.type`, the ECDSA/EdDSA signature over `authenticatorData || SHA-256(clientDataJSON)` and the Level 3-specific `signCount` policy (see Section 5.8).

- Fifth, a **software authenticator emulator** is provided so that the full test suite runs without hardware dependencies.

- Sixth, a **minimal client page** to demonstrated the browser side integration, including discoverable credentials and conditional mediation autofill.

5.3 Technology Stack

The server side prototype uses Python 3.11. The following libraries are employed:

- `cryptography` v42 for ECDSA P-256 and Ed25519 signature verification, EC key parsing and SHA-256.

- `cbor2` v5 for CBOR decoding of the `attestationObject` envelope and COSE_Key structures.
- `flask` v3 as a lightweight HTTP server for the RP endpoints.
- `secrets` and `os` for cryptographically random challenge and user handle generation.
- `base64` and `json` for encoding and decoding ceremony payloads.
- `pytest` v8 for the automated test suite.

The client side interface is a basic HTML login page using the native browser WebAuthn API.
And also anothor HTML dashboard page to act as a validation page, simulating that the login was successful.

5.4 System Architecture

The prototype is organised into **three Python layers** with strict separation of responsabilities, as we defined previously in the Section 4.2.

The `parsing.py` contains every routine that handles raw bytes: base64url encoding and decoding, the `authenticatorData` layout decoder, the COSE_Key decoder that produces a `cryptography` public key object and the ECDSA/EdDSA signature verification wrapper.

The `ceremonies.py` implements the pure verification logic for both ceremonies. its functions take every input as a parameter and return either a persistent credential record (registration) or an assertion outcome (authentication) to the caller.

The `storage.py` holds the in memory state for users, credentials and pending challenges. Challenges are single use and time limited (120 s TTL, the value adopted in the design of Section 4.2).

The `app.py` is the Flask application that wires HTTP to the ceremony layer.
It's responsible for exposes the endpoints:

- `/api/registration/options`,
- `/api/registration/verify`,
- `/api/authentication/options`,
- `/api/authentication/verify`,
- `/api/me`,
- `/api/logout`
- and `/api/credentials/<id>/delete`

Sessions are stored in signed cookies marked `HttpOnly`, `SameSite=Strict` and `Secure`.

The `static/` folder containg all the client side: the HTML pages, the Javascript files and the CSS.

The `login.html` that drives `navigator.credentials.create()` and `.get()`, supports explicit and discoverable authentication and arms conditional mediation autofill on page load.

And also the `dashboard.html` that simulates a post-login dashboard view that the login flow redirects to on success.

The `emulator.py` implements the software authenticator.

And the `test_webauthn.py` contains both the security property matrix and the supplementary coverage (see Section 5.10).

A small script was created, `scripts/deploy-poc.sh`, to brings the RP up behind a Cloudflare quick tunnel with the relying party identity relinked to the public HTTPS host.

This is the integration vehicle for the cross device scenario explained in the Section 4.5.

5.5 Core Data Structures

```
# storage.py

CHALLENGE_TTL_SECONDS = 120
CHALLENGE_BYTES = 32

@dataclass
class Credential:
    """A credential registered for a user account."""

    credential_id: bytes # the unique ID of the credential
    user_id: bytes # the user who owns it
    public_key_cose: dict[int, Any] # the public key in COSE format
    sign_count: int   # the counter used in WebAuthn
    be: bool          # backup eligible
    bs: bool          # backup state
    aaguid: bytes     # authenticator model identifier
    transports: list[str]   # how the credential can be used,
    created_at: float   # when it was created
    nickname: str = ""  # optional friendly name

    @property
    def is_synced(self) -> bool:
        """
        Treat BE=1 as "synced or syncronable" for the signCount policy.
        (WebAuthn L3 API docs - Section 6.1.1).
        """
        return self.be
```

```
@dataclass
class User:
    """A user account.

    'user_id' is the WebAuthn 'user.id' (random 16 bytes)
        and is what the authenticator stores as the user handle.
    'username' is the human-typed identifier and serves as
        the lookup key in the '_users_by_name' index.
    """

    user_id: bytes
    username: str
    display_name: str

@dataclass
class _PendingChallenge:
    """A challenge issued by the server but not yet consumed."""

    purpose: str
    created_at: float
    username: str | None

# ---- In memory table
_users: dict[bytes, User] = {}                        # maps user_id to a User
_users_by_name: dict[str, bytes] = {}                 # maps username to user_id
_credentials: dict[bytes, Credential] = {}            # maps credential_id to a Credential
_challenges: dict[bytes, _PendingChallenge] = {}      # maps challenge bytes to a
                                                          pending challenge record
```

First of all, we have defined two variables relating to the challenge properties: the **challenge live time** (TTL) and the **number of random bytes of the challenge**, respectively.

Next, we have the `Credential` dataclass which represents one registered passkey or WebAuthn credential with its relevant attributes, which are self explanatory in the code comments.

Then the `be` and `bs` fields on `Credential` class, respectively are extracted from `authenticatorData` object.

And the property function `is_synced` checks the `signCount` policy.

Below, we define the `User` dataclass and also the `PendingChallenge` class that represents a challenge that the server created but the user has not used yet.

The challenge store is keyed by the 32 byte random challenge itself.

Each entry records its *purpose* (registration vs authentication) so that a challenge issued for one ceremony cannot be replayed as the other.

A TTL of 120 s (matching the value fixed in the design of Section 4.2) is enforced on every read, that expired entries are evicted opportunistically by `_gc_challenges()` on each call to

`create_challenge` or `consume_challenge`, so the store cannot grow unbounded if a client abandons a ceremony.

Selecting the upper bound is a deliberate usability choice rather than a security relaxation, because the replay attack is already eliminated by the atomic single use semantics of `consume_challenge`, so the TTL governs only how long the server retains a pending but unused challenge, not the duration of any replay window.

The storage itself are the in memory "tables" using dictionaries, this is the heart of the prototype storage.

The relying party configuration lives in a single dataclass that is used across all the ceremony function.

```
# ceremonies.py

@dataclass
class RpConfig:
    """Relying-Party identity and policy flags.

    ‘origin‘ is matched exactly.
    ‘rp_id‘ is the domain scope.
    ‘reject_backup_eligible‘ refuses synced credentials (hardware-bound only).
    """

    rp_id:                    str
    rp_name:                  str
    origin:                   str
    require_user_verification: bool = True
    reject_backup_eligible:    bool = False
```

The `origin` field is matched byte for byte against the value the browser writes into `clientDataJSON` and this is the exact mechanism by which WebAuthn provides phishing resistance.

The `require_user_verification` flag is left at its strict default value, so that every assertion must carry the UV bit, relaxing this would cause the prototype to switch to a presence only authentication mode, outside the Level 3 passkey specifications.

The `reject_backup_eligible` switch is the policy hook discussed in Section 4.3, a deployment that demands hardware-bound assurance can set it to `True` and refuse synced credentials at registration time.

The same `RpConfig` instance is consulted by the `verify_registration` and the `verify_authentication` functions, so the policy is applied uniformly across both ceremonies.

Finally, after reviewing and understanding the data flow of the prototype, it could be summarised graphicaly using the following diagram.

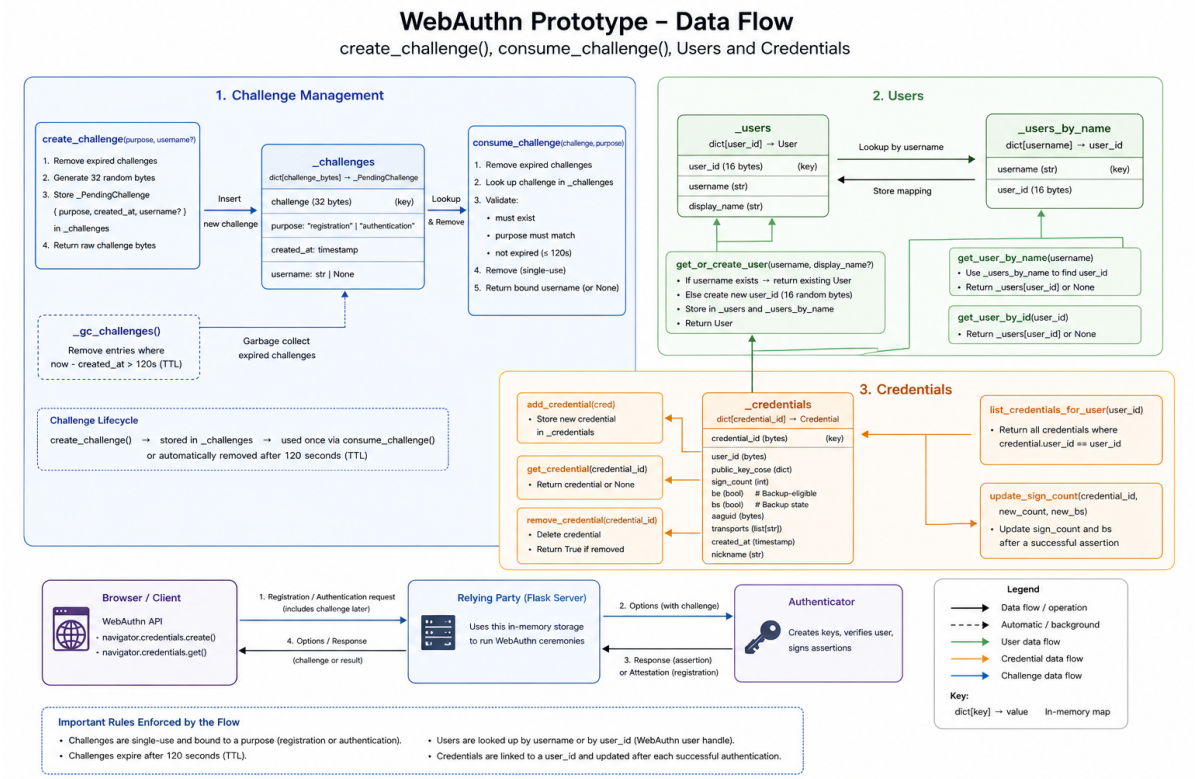


Figure 5.1: Diagram of the Data Flow of the Prototype.

5.6 Challenge Generation

The `/api/registration/options` endpoint assembles a `PublicKeyCredentialCreationOptions` dictionary, as we seen earlier in the Section 3.2.1.

```
@app.post("/api/registration/options")
def registration_options():
    body = request.get_json(silent=True) or {}
    username = (body.get("username") or "").strip()
    if not username:
        return jsonify({"error": "username required"}), 400

    user = storage.get_or_create_user(
        username, body.get("displayName") or username
    )
    challenge = storage.create_challenge("registration", username)

    exclude = [
        {"type": "public-key",
         "id":   b64url_encode(c.credential_id),
         "transports": c.transports}
```

```
        for c in storage.list_credentials_for_user(user.user_id)
    ]

    return jsonify({
        "rp":   {"id": RP.rp_id, "name": RP.rp_name},
        "user": {
            "id":          b64url_encode(user.user_id),
            "name":        user.username,
            "displayName": user.display_name,
        },
        "challenge": b64url_encode(challenge),
        "pubKeyCredParams": [
            {"type": "public-key", "alg": -7},   # ES256
            {"type": "public-key", "alg": -8},   # EdDSA
        ],
        "timeout": WEBAUTHN_TIMEOUT_MS,          # = 120_000 ms
        "attestation": "none",
        "authenticatorSelection": {
            "residentKey":      "required",    # discoverable credential
            "userVerification": "required",
        },
        "excludeCredentials": exclude,
    })
```

The server issues a fresh 32-byte challenge from `secrets.token_bytes` on every call, the value satisfies the WebAuthn L3 entropy requirement that is at least 16 bytes, in our case we define 32 bytes as we shown in the Section 5.5.

The hardware-bound or the passkey choices appear here.
The `residentKey` parameter is set to `required` so that the authenticator creates a *discoverable credential* and the server can resolve the user from the credentialId alone on subsequent sign in.

The `excludeCredentials` list prevents the authenticator from silently overwriting a credential the same user has already enrolled (WebAuthn API documetation Section 5.1.3 [W3C26]).

The `attestation` parameter is fixed to `none` so that the browser don't raise the privacy prompt UI, the prototype contribution is the structural verification path, not attestation-CA management and the `packed` self attestation path is exercised separately from a hand crafted attestation statement.

The `clientDataJSON` envelope returned by the browser is decoded and validated by a private helper that both ceremonies share, ensuring identical handling of `type`, `challenge`, `origin` and `crossOrigin`.

```
# ceremonies.py

def _verify_client_data(
    client_data_json:       bytes,
```

```
    expected_type:          str,
    expected_challenge:     bytes,
    rp:                     RpConfig,
) -> None:
    """Validate the clientDataJSON envelope. Raises ValueError on any mismatch."""
    client_data = json.loads(client_data_json)

    ctype = client_data.get("type")
    if ctype != expected_type:
        raise ValueError(f"clientData.type mismatch: got {ctype}, expected {expected_type}"

    if b64url_decode(client_data.get("challenge")) != expected_challenge:
        raise ValueError("challenge mismatch")

    origin = client_data.get("origin")
    if origin != rp.origin:
        raise ValueError(f"origin mismatch: got {origin}, expected {rp.origin}")
```

The same routine is invoked from `verify_registration` with `expected_type="webauthn.create"` and from `verify_authentication` with `expected_type="webauthn.get"`, so a registration challenge presented during an authentication attempt fails the type check before any cryptographic work is performed.

The `crossOrigin` guard refuses assertions which top level browsing context don't match the relying party origin: the WebAuthn specification permits cross-origin assertions in principle, but every additional cross-origin path expands the attack surface and the prototype rejects them entirely.

5.7 Registration Verification

The registration verification is split across `parsing.parse_authenticator_data`, which decodes the structure, and `ceremonies.verify_registration`, which enforces the policy.

The parsing helper is listed first:

```
# parsing.py

FLAG_UP = 1 << 0    # User Present
FLAG_UV = 1 << 2    # User Verified
FLAG_BE = 1 << 3    # Backup Eligible  (Level 3 addition)
FLAG_BS = 1 << 4    # Backup State     (Level 3 addition)
FLAG_AT = 1 << 6    # Attested Credential Data present
FLAG_ED = 1 << 7    # Extension Data present

# Fixed-prefix layout constants (WebAuthn L3 Section 6.1, Figure 4).
_AUTH_DATA_HEADER_LEN      = 37     # rpIdHash(32)+flags(1)+signCount(4)
_AAGUID_LEN                = 16
_CRED_ID_LEN_FIELD_LEN     = 2
```

```
_ATTESTED_CRED_DATA_HEADER = 55     # = HEADER + AAGUID + len-field

_EC_COORDINATE_LEN = 32               # P-256 x and y are 32 bytes each
_ED25519_PUBLIC_KEY_LEN = 32          # raw Ed25519 public key length

@dataclass
class AuthData:
    """Structured view of the authenticatorData blob."""

    rp_id_hash: bytes
    flags: int
    sign_count: int
    # Only present when the AT flag is set (registration ceremonies).
    aaguid: bytes | None = None
    credential_id: bytes | None = None
    credential_public_key: dict[int, Any] | None = None  # COSE key as a dict

    # Convenience flag accessors - make call-site code self-explanatory.
    @property
    def up(self) -> bool:
        """User-Present flag: the user interacted with the authenticator."""
        return bool(self.flags & FLAG_UP)

    @property
    def uv(self) -> bool:
        """User-Verified flag: biometric / PIN check succeeded."""
        return bool(self.flags & FLAG_UV)

    @property
    def be(self) -> bool:
        """Backup-Eligible flag: credential is multi device (synced passkeys)."""
        return bool(self.flags & FLAG_BE)

    @property
    def bs(self) -> bool:
        """Backup-State flag: credential is currently synced."""
        return bool(self.flags & FLAG_BS)

    @property
    def at(self) -> bool:
        """Attested-credential-data flag: set on registration, cleared on assertion."""
        return bool(self.flags & FLAG_AT)


def parse_authenticator_data(data: bytes) -> AuthData:
    """Parse the fixed+variable length authenticatorData structure."""
    auth = AuthData(
```

```
        rp_id_hash=data[0:32],
        flags=data[32],
        sign_count=int.from_bytes(data[33:_AUTH_DATA_HEADER_LEN], "big"),
    )
    if not auth.at:
        return auth

    aaguid_end = _AUTH_DATA_HEADER_LEN + _AAGUID_LEN  # 53
    auth.aaguid = data[_AUTH_DATA_HEADER_LEN:aaguid_end]
    cred_id_len = int.from_bytes(
        data[aaguid_end:_ATTESTED_CRED_DATA_HEADER], "big",
    )
    start = _ATTESTED_CRED_DATA_HEADER
    end = start + cred_id_len
    auth.credential_id = data[start:end]

# ceremonies.py

def verify_registration(
    attestation_object_b64: str,
    client_data_json_b64: str,
    expected_challenge: bytes,
    rp: RpConfig,
    user_id: bytes,
) -> Credential:
    """Verify a registration ceremony and return a ready-to-store Credential.
        Raises ValueError on any validation failure.
    """
    # Decode inputs.
    client_data_json = b64url_decode(client_data_json_b64)
    attestation_object = b64url_decode(attestation_object_b64)

    # Validate clientDataJSON envelope.
    _verify_client_data(
        client_data_json,
        expected_type="webauthn.create",
        expected_challenge=expected_challenge,
        rp=rp,
    )

    # Decode the CBOR attestationObject.
    att_obj = cbor2.loads(attestation_object)
    fmt = att_obj.get("fmt")
    auth_data_bytes = att_obj.get("authData")
    att_stmt = att_obj.get("attStmt") or {}

    if fmt not in SUPPORTED_ATTESTATION:
```

```
    raise ValueError(f"unsupported attestation format: {fmt!r}")

auth_data = parse_authenticator_data(bytes(auth_data_bytes))

# rpIdHash must equal SHA-256(rp.rp_id).
expected_rp_id_hash = hashlib.sha256(rp.rp_id.encode("utf-8")).digest()
if auth_data.rp_id_hash != expected_rp_id_hash:
    raise ValueError("rpIdHash mismatch")

# Required flags on registration.
if not auth_data.up:
    raise ValueError("UP (user present) flag not set")
if rp.require_user_verification and not auth_data.uv:
    raise ValueError("UV (user verified) flag required but not set")

# Policy: refuse synced credentials when hardware-bound is required.
if rp.reject_backup_eligible and auth_data.be:
    raise ValueError("policy: backup-eligible (multi-device) credentials rejected")

# cose_to_public_key gates on (kty, alg); unsupported combos raise here.
cose_key = auth_data.credential_public_key
alg = cose_key.get(COSE_KEY_LABEL_ALG)
public_key = cose_to_public_key(cose_key)

if fmt == "packed":
    _verify_packed_attestation(
        att_stmt=att_stmt,
        auth_data_bytes=bytes(auth_data_bytes),
        client_data_json=client_data_json,
        credential_public_key=public_key,
        credential_alg=alg,
    )
# fmt == "none": nothing to verify.

log.info(
    "registration verified: alg=%d fmt=%s be=%s bs=%s",
    alg, fmt, auth_data.be, auth_data.bs,
)

return Credential(
    credential_id=auth_data.credential_id,
    user_id=user_id,
    public_key_cose=cose_key,
    sign_count=auth_data.sign_count,
    be=auth_data.be,
    bs=auth_data.bs,
    aaguid=auth_data.aaguid,
```

```
        transports=[],
        created_at=time.time(),
    )
```

Two additions are specific to Level 3. First, the `be` and `bs` fields are captured from `authenticatorData` and persisted, so the server can later identify synchronised passkeys and apply the relaxed `signCount` policy of Section 5.8.

Second, the optional `reject_backup_eligible` policy allows deployments with a hardware-bound model to refuse multi-device credentials at registration time.

Below is a diagram showing what the registration process might look like, focusing on how the prototype is structured.

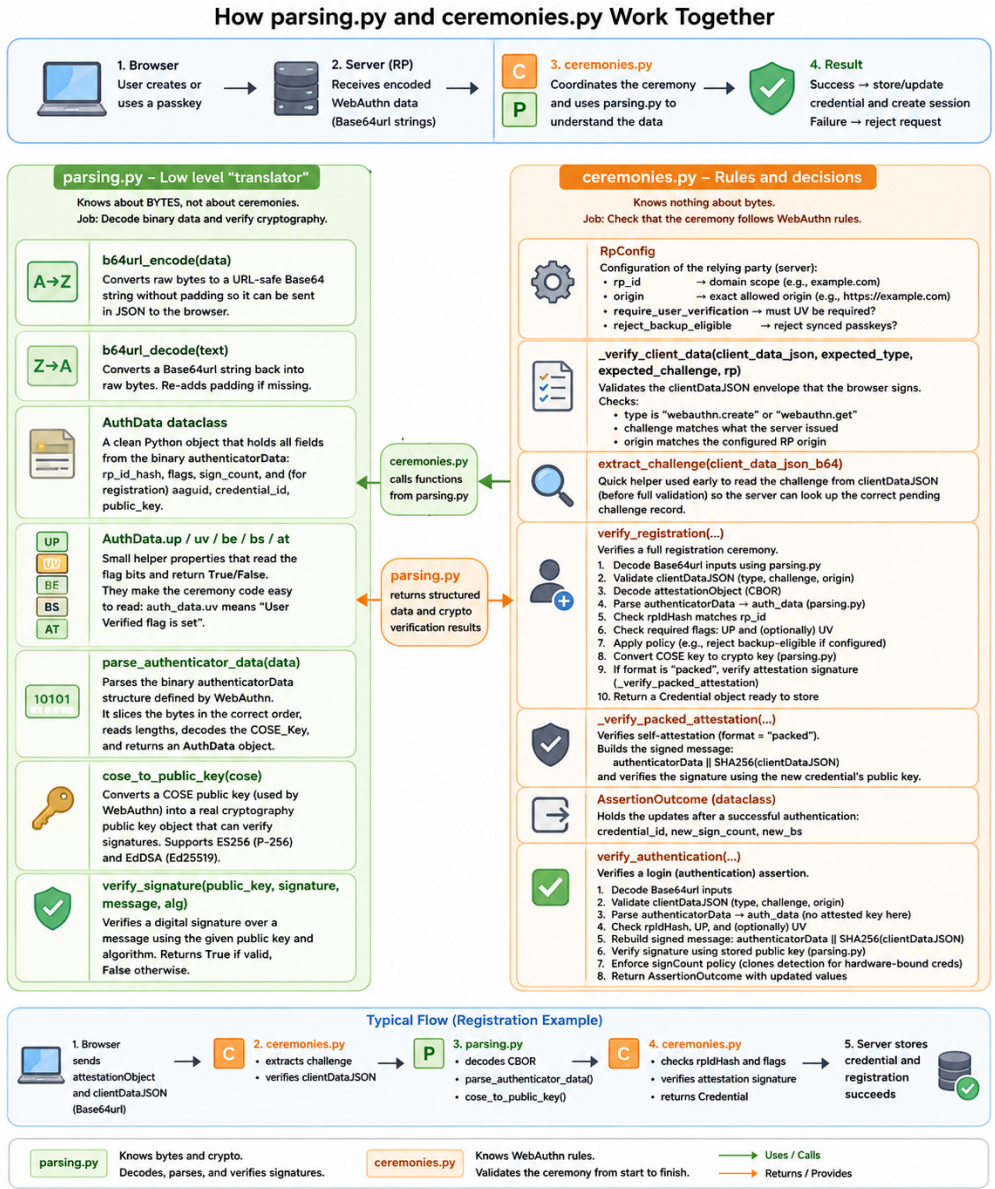


Figure 5.2: Diagram of the Registration Flow of the Prototype.

5.8 Authentication Verification

```
# ceremonies.py

def verify_authentication(
    credential_id: bytes,
    authenticator_data_b64: str,
    client_data_json_b64: str,
    signature_b64: str,
    expected_challenge: bytes,
    rp: RpConfig,
    stored: Credential,
) -> AssertionOutcome:
    """Verify a single authentication assertion.

    Returns an AssertionOutcome with values to persist. Does not mutate 'stored'.
    Raises ValueError on any validation failure.
    """
    # Decode inputs.
    client_data_json = b64url_decode(client_data_json_b64)
    auth_data_bytes = b64url_decode(authenticator_data_b64)
    signature = b64url_decode(signature_b64)

    # Validate clientDataJSON envelope.
    _verify_client_data(
        client_data_json,
        expected_type="webauthn.get",
        expected_challenge=expected_challenge,
        rp=rp,
    )

    auth_data: AuthData = parse_authenticator_data(auth_data_bytes)

    # rpIdHash and required flags.
    expected_rp_id_hash = hashlib.sha256(rp.rp_id.encode("utf-8")).digest()
    if auth_data.rp_id_hash != expected_rp_id_hash:
        raise ValueError("rpIdHash mismatch")
    if not auth_data.up:
        raise ValueError("UP flag not set")
    if rp.require_user_verification and not auth_data.uv:
        raise ValueError("UV flag required but not set")

    # Verify the assertion signature over (authenticatorData || SHA-256(clientDataJSON)).
    public_key = cose_to_public_key(stored.public_key_cose)
    alg = stored.public_key_cose.get(COSE_KEY_LABEL_ALG)
    client_data_hash = hashlib.sha256(client_data_json).digest()
    message = auth_data_bytes + client_data_hash
```

```
    if not verify_signature(public_key, signature, message, alg):
        raise ValueError("assertion signature is invalid")

    # signCount policy: monotonic for hardware-bound credentials only.
    # Synced credentials may legitimately report 0 every time.
    if not stored.is_synced:
        counter_used = auth_data.sign_count > 0 or stored.sign_count > 0
        if counter_used and auth_data.sign_count <= stored.sign_count:
            raise ValueError(
                f"signCount did not increase ({auth_data.sign_count} <= "
                f"{stored.sign_count}); possible cloned authenticator"
            )

    log.info(
        "authentication verified: alg=%d synced=%s sign_count=%d→%d",
        alg, stored.is_synced, stored.sign_count, auth_data.sign_count,
    )

    return AssertionOutcome(
        credential_id=credential_id,
        new_sign_count=auth_data.sign_count,
        new_bs=auth_data.bs,
    )
```

The `signCount` branch implements the subtile Level 3 rule, for a synchronised passkey (`is_synced`, i.e., `BE=1`), the counter is useless because the private key is replicated across devices that increment independently between them. The server must accept any reported value, including zero on every assertion.

Only hardware-bound credentials (`BE=0`) trigger the monotonic counter check and a regression is treated as evidence of a cloned authenticator.

The signature is verified with `public_key.verify()` from `cryptography`.
The `verify_signature` wrapper in `parsing.py` reports cryptographic mismatch as a single Boolean return, so the ceremony code can express the check with a simple (if not ..) conditional.

The message format `authData || SHA-256(clientDataJSON)` is the WebAuthn construction that binds every assertion to its challenge, origin and `rpIdHash` simultaneously.

5.9 Software Authenticator Emulator

The emulator (`emulator.py`) emits identical bytes of the `attestationObject` and assertion structures to what a platform authenticator would produce, using an ECDSA P-256 key pair.

Two modes are supported: synchronised (`backup_eligible=True`) and hardware-bound (`backup_eligible=False`), so that every Level 3 policy branch can be exercised from tests.

```
# emulator.py

class Authenticator:
    """Software authenticator that holds one or more credentials.

    backup_eligible=True emulates a synced platform authenticator (BE/BS set,
    signCount stays at 0). False emulates a device-bound key whose signCount
    increments on each assertion.
    """

    def __init__(self, aaguid: bytes = b"\x00" * 16, backup_eligible: bool = True):
        """Create an empty authenticator. Credentials are added by create()."""
        self.aaguid = aaguid
        self.backup_eligible = backup_eligible
        self._credentials: dict[bytes, dict] = {}

    # ----- Registration ceremony -----

    def create(
        self,
        *,
        rp_id: str,
        challenge: bytes,
        origin: str,
        user_id: bytes,
        user_verified: bool = True,
    ) -> dict:
        """Emulate navigator.credentials.create() and return a PublicKeyCredential dict."""
        private_key = ec.generate_private_key(ec.SECP256R1())
        nums = private_key.public_key().public_numbers()
        cose_key = {
            1: 2,                               # kty = EC2
            3: -7,                              # alg = ES256
            -1: 1,                              # crv = P-256
            -2: nums.x.to_bytes(32, "big"),
            -3: nums.y.to_bytes(32, "big"),
        }

        credential_id = os.urandom(32)
        self._credentials[credential_id] = {
            "private_key": private_key,
            "sign_count": 0,
            "user_id": user_id,
        }

        flags = FLAG_UP | FLAG_AT
        if user_verified:
```

```
        flags |= FLAG_UV
    if self.backup_eligible:
        flags |= FLAG_BE | FLAG_BS

    auth_data = self._build_auth_data(
        rp_id=rp_id,
        flags=flags,
        sign_count=0,
        attested=(self.aaguid, credential_id, cose_key),
    )
    attestation_object = cbor2.dumps({
        "fmt": "none",
        "attStmt": {},
        "authData": auth_data,
    })
    client_data = self._client_data("webauthn.create", challenge, origin)

    return {
        "id": b64url_encode(credential_id),
        "rawId": b64url_encode(credential_id),
        "type": "public-key",
        "response": {
            "attestationObject": b64url_encode(attestation_object),
            "clientDataJSON":    b64url_encode(client_data),
            "transports": ["internal"],
        },
    }
```

The emulator generate `fmt=none` attestations because the `packed` verification path in `ceremonies.py` runs separately from a manually attestation statement in the unit tests.

5.10 Security Property Test Suite

The prototype designed to validate the security properties enumerated in Section 4.4 and other basic operating properties.

The test suite are defined in a single file, `test_webauthn.py`,

Every test use the full Flask application through `app.test_client()` and the software authenticator of Section 5.9, without mocking the ceremony layer.

The matrix in Table 5.1 groups the validation into three classes:

- The **T1**–**T2** confirm the positive path: a synced passkey registers correctly and a valid ES256 signature opens a session.
- The **T3**–**T6** cover the four orthogonal forgery defences — challenge replay, origin substitution, signature tampering, and rpId substitution — each exercised by feeding the server an otherwise well-formed assertion with a single invalidating change.

- The **T7–T11** validate the Level 3-specific logic: the monotonic-counter check on device-bound credentials (T7), the explicit acceptance of `signCount=0` on every assertion for synced credentials (T8), the parsing and persistence of the BE flag (T9 and T10), and the optional RP policy that refuses multi-device credentials when a hardware-bound threat model is in force (T11).

ID	Test function	Security property
T1	`test_passkey_registration_success`	Credential creation; BE flag detection
T2	`test_passkey_auth_success`	ES256 signature verification
T3	`test_replay_rejected`	Replay prevention (single use challenge)
T4	`test_phishing_rejected`	Phishing resistance (origin binding)
T5	`test_tampered_signature_rejected`	Signature integrity
T6	`test_wrong_rpid_rejected`	Domain binding via rpIdHash
T7	`test_hw_clone_detection`	Device-bound signCount enforcement
T8	`test_passkey_zero_count_accepted`	Level 3 passkey counter exemption
T9	`test_be_flag_marks_synced`	Level 3 flag parsing (synced)
T10	`test_hw_not_synced`	BE=0 for hardware-bound credentials
T11	`test_rp_rejects_synced_when_hw_required`	RP credential type policy

Table 5.1: The security property matrix implemented by `test_webauthn.py`.

The methodology and exemplar test code are presented in Section 6.1.

6 Evaluation and Results

6.1 Test Suite Design

The automated test suite exercises the server side verification logic and validates the security properties enumerated in Section 4.4.

The test suite consists of two files:

- the `emulator.py`, which provides the software authenticator described in Section 5.9.
- the `test_webauthn.py`, which contains both the eleven test securityproperty matrix (Table 5.1) and two complementary tests covering the discoverable credential flow and the `/api/me` session endpoint.

Each test run the full Flask application via `app.test_client()` and the ceremony layer is never mocked.

Then we do a fixture that wipes the in memory storage before and after each test so registrations made by one test cannot leak into the next.

Two driver helpers encapsulate a full ceremony round trip. In every test in the matrix is built on top of them.

The helpers accept a parameterised `origin` and `rp_id` so that origin substitution and rpId substitution attacks can be modelled by changing a single argument.

The `mutate_signature` similarly enables signature tampering tests by injecting a callable that corrupts the raw bytes between signing and submission.

```
# test_webauthn.py (driver helpers)

ORIGIN = "http://localhost:5000"
RP_ID  = "localhost"

@pytest.fixture(autouse=True)
def _reset_state():
    storage.reset_all()
    yield
    storage.reset_all()


@pytest.fixture
def client():
    app.app.testing = True
```

```
    return app.app.test_client()


def _register(client, authenticator, username="alice", *, origin=ORIGIN):
    opts = client.post("/api/registration/options",
                       json={"username": username}).get_json()
    cred = authenticator.create(
        rp_id     = opts["rp"]["id"],
        challenge = b64url_decode(opts["challenge"]),
        origin    = origin,
        user_id   = b64url_decode(opts["user"]["id"]),
    )
    return client.post("/api/registration/verify",
                       json={"username": username, "credential": cred})


def _authenticate(client, authenticator, username="alice",
                  *, origin=ORIGIN, rp_id=RP_ID,
                  credential_id=None, mutate_signature=None):
    opts = client.post("/api/authentication/options",
                       json={"username": username}).get_json()
    cred = authenticator.get(
        rp_id         = rp_id,
        challenge     = b64url_decode(opts["challenge"]),
        origin        = origin,
        credential_id = credential_id,
    )
    if mutate_signature is not None:
        sig = bytearray(b64url_decode(cred["response"]["signature"]))
        mutate_signature(sig)
        cred["response"]["signature"] = b64url_encode(bytes(sig))
    return client.post("/api/authentication/verify",
                       json={"credential": cred})
```

Three representative tests show how the helpers compose.

The test T2 checks the **positive authentication path**, that follows a synced credential is registered and a fresh assertion opens a session.

```
def test_passkey_auth_success(client):
    auth = Authenticator(backup_eligible=True)
    _register(client, auth)
    r = _authenticate(client, auth)
    assert r.status_code == 200
    assert r.get_json()["username"] == "marc"
```

The test T4 replace the origin to **simulate a phishing site**.

The server is expected to reject the assertion because the `origin` signed into `clientDataJSON` no matches with the relying party expected origin as you can see in the test:

```
def test_phishing_rejected(client):
    auth = Authenticator(backup_eligible=True)
    _register(client, auth)
    r = _authenticate(client, auth, origin="http://evil.example")
    assert r.status_code == 400
    assert "origin" in r.get_json()["error"].lower()
```

The test T8 verifies the Level 3 `signCount` exemption, as we explained earlier in the Section 3.7.3.

A synced credential reports `signCount=0` on every assertion and the server must accept this indefinitely.

We execute a loop of five round trips to make explicit that the value is allowed to repeat, as can see in the following code:

```
def test_passkey_zero_count_accepted(client):
    auth = Authenticator(backup_eligible=True)
    _register(client, auth)
    for _ in range(5):
        r = _authenticate(client, auth)
        assert r.status_code == 200
```

The remaining matrix tests follow the same composition pattern, their assertions are summarised in Table 5.1.

6.2 Test Results and Prototype Validation

All thirteen tests have the expected results, the eleven matrix tests of Table 6.1 plus the two complementary tests covering the discoverable credential flow and the `/api/me` endpoint.

Table 6.1: Prototype test results and validated security properties.

ID	Test Case	Security Property	Expected	Result
T1	Valid passkey registration	Credential creation; BE flag detection	Pass	✓
T2	Valid passkey authentication	ES256 signature verification	Pass	✓
T3	Replay attack	Replay prevention	Reject	✓
T4	Wrong origin (phishing)	Phishing resistance	Reject	✓
T5	Tampered signature	Signature integrity	Reject	✓
T6	Wrong rpId	Origin/domain binding	Reject	✓
T7	HW clone detection	signCount enforcement	Reject	✓
T8	Passkey `signCount=0`	Level 3 passkey counter spec	Accept	✓
T9	BE flag marks synced	Level 3 flag parsing	Pass	✓
T10	HW credential not synced	BE = 0 for hardware key	Pass	✓
T11	RP policy: HW-only context	RP credential type policy	Reject	✓

The test results confirm the key theoretical claims of the security analysis.

The tests **T1**–**T2** establish that the ceremony verification logic correctly validates well formed registration and authentication responses, including the Level 3 BE/BS flag parsing.

The tests **T3**–**T6** confirm that phishing, replay, signature tampering and wrong origin attacks are structurally rejected by the server side verification algorithm.

The test **T7** confirms that hardware-bound credentials correctly enforce `signCount` monotonicity as a clone detection mechanism.

The tests **T8**–**T10** validate the Level 3 distinction that form the core of the passkey model: synchronised passkeys legitimately report `signCount=0` on every assertion and must be accepted without triggering the clone detection error, while hardware-bound credentials continue to enforce monotonicity.

The test **T11** exercises the policy enforcement mechanism by which an RP operating a high assurance context can reject backup eligible credentials at registration time, restricting acceptance only to hardware-bound credentials.

These results provide empirical grounding for the theoretical claims of Chapter 4, particularly the assessment that phishing resistance and replay prevention are preserved in the passkey model while clone detection is structurally unavailable for synchronised credentials.

6.3 Evaluation Criteria Framework

The comparative evaluation of Section 6.4 rates six authentication mechanisms against sixteen criteria grouped into four categories: *security*, *usability*, *deployment* and *privacy*.

The categories follow the structure used by the academic literature on web authentication [Bon+12; Lya+20], adapted to the WebAuthn Level 3 specification in which credentials may be hardware-bound or synchronised across user devices.

Each criterion is defined below, together with what is being measured and the source of the rating shown in Table 6.2.

6.3.1 Security Criteria

The security category captures the resistance to the attack classes that drive real world credential compromise.

Phishing resistance. Measures whether the protocol prevents an attacker who controls a fraudulent origin from obtaining a valid credential.

The rating reflects whether origin information remains linked to the authentication response from an entity that the user cannot override (the browser writing the actual toplevel origin into `clientDataJSON`, in the WebAuthn case).
Validated by test T4.

Replay prevention. Measures whether a captured authentication exchange can be resubmitted to the server and accepted as a fresh login.

The rating indicates whether each attempt is linked to a fresh, server issued, single use value that the server invalidates before the next attempt.
Validated by test T3.

Server breach exposure. Measures the operational impact of a relying party database breach in which the credential store is exfiltrated.

The rating reflects what an offline attacker can recover from the stolen records: cleartext passwords (*High*), salted password hashes (*High* after a successful cracking campaign), shared OTP seeds (*High*) or only public keys with no shared secret (*Minimal*).

MITM resistance. Measures resistance to an active attacker who relays traffic between the user and the legitimate server.

The rating indicates whether the cryptographic link between client and server is bound to the actual TLS endpoint, so a relayed exchange cannot be presented again to the legitimate server.
In WebAuthn this property is delivered by the same origin binding mechanism that delivers phishing resistance.

Clone detection. Measures whether the protocol can detect that a credential's private material has been duplicated and is being used in parallel by an attacker.

The rating reflects whether the authenticator maintains state (typically a monotonic `signCount`) whose divergence between legitimate device and clone would surface to the server.
As discussed in Section 4.3 and validated by tests T7–T8, this property is available for hardware-bound WebAuthn credentials but is structurally absent for synchronised passkeys because the WebAuthn Level 3 specification permits `signCount = 0` on every assertion for synchronised credentials, so a clone is indistinguishable from legitimate cross device use.

Platform account risk. Measures the additional risk that the authentication mechanism inherits from a third party identity provider or platform account.

The rating reflects how much of the credential effective security depends on the security of an account the relying party does not control: the user's federation provider for federated login (*High*) or the platform account managing passkey synchronisation (Apple ID, Google Account, Microsoft Account) for synchronised passkeys (*Moderate*).
Hardware-bound credentials are unaffected by platform account compromise (*Low*).

6.3.2 Usability Criteria

The usability category captures the cost the mechanism imposes on legitimate users during routine authentication and recovery.

Interaction cost. Measures the cognitive and physical effort a typical sign in requires, counting steps, devices and context switches.

A password entered in a single field rates *Low*, an OTP that requires consulting a second device and copying a code rates *Moderate*, a passkey approved with a single biometric gesture rates *Low*.

Accessibility. Measures the proportion of the users who can complete the authentication process regardless of any impairments, device type or context.

The rating reflects hard prerequisites the mechanism imposes on the user: typing accuracy, vision, fine motor control, smartphone ownership or possession of a secondary device.

Mechanisms that fall back gracefully to alternative modalities rate higher than those that require a single specific interaction pattern.

Account recovery. Measures the viability of regaining access after losing the primary authentication factor.

The rating reflects what fallback paths the mechanism supports and how much manual operator intervention they require.

The passwords have *Simple* recovery (email reset link).

The hardware-bound WebAuthn has *Complex* recovery (a lost authenticator with no pre-registered backup typically forces a high friction identity proofing process).

The synchronised passkeys are *Improved* relative to hardware-bound because the credential is recoverable on any device that signs into the same platform account, at the cost of the platform account dependency captured in the security category.

6.3.3 Deployment Criteria

The deployment category covers the operational and engineering investment required to support the mechanism in a real relying party deployment.

Server implementation cost. Measures the engineering effort the relying party must invest to support the mechanism.

The rating reflects the volume and complexity of the code that the RP must execute, such as the ceremony options issuance, attestation and assertion verification, signCount policy, BE/BS handling and policy hooks.

The WebAuthn rates *High* because an RP must either implement these checks directly (the approach taken by this prototype) or accept the operational cost of integrating and maintaining a third party verification library.

Third party dependency. Measures the extent to which the authentication flow depends on services controlled by other parties at runtime, that is, after a user attempts to sign in.

The federated login rates *High* because every sign in involves a redirect to the IdP and fails if the IdP is unreachable.

The synchronised passkeys rate *Moderate* because the cross device hybrid transport uses provider's' tunnel servers for QR based handoff, while the standard flow on the same device has no such dependency.

The hardware-bound WebAuthn rates *Low* because the verification path is entirely server side.

Browser support. Measures the range of browser and platform combinations that can use the mechanism without provider specific configuration.

The rating is taken from the major browser baseline as of early 2026: Chrome, Firefox, Safari and Edge on their primary platforms.

Legacy compatibility. Measures the ability to coexist with older clients, older servers or accounts created before the mechanism was implemented.

The rating reflects whether existing users can continue to authenticate while the new mechanism is being implmemented, which is an essential requirement for any consumer facing migration.

The WebAuthn rates *Moderate* because deployment is gated on browser and authenticator availability and almost always coexists with a password fallback during the transition.

6.3.4 Privacy Criteria

The privacy category captures the extent to which the mechanism limits the personal information collected by the relying party.

Server data minimisation. Measures the volume and sensitivity of personal data the relying party must store to operate the mechanism.

The rating summarises what an external observer of the database would learn about the user beyond what is strictly necessary for authentication: a salted password hash plus profile data (*Hash+PII*), an OTP shared secret plus profile data (*Sec+PII*), an externally maintained federation profile (*Profile*) or a public key with no shared secret (*PK only*).

Anonymity support. Measures whether authenticating across services enables a third party to correlate the same user across origins.

The rating reflects whether the mechanism produces identifiers that are linkable across origins. The WebAuthn delivers per origin key isolation by design, because the credential generated for origin A is distinct from the credential for origin B and the public keys carry no cross origin correlation.

The synchronised passkeys preserve this property at the WebAuthn layer, but the moderate rating reflects the indirect correlation introduced by the platform account that synchronises the credentials.

GDPR alignment. Measures how easily a deployment of this mechanism can comply with the European General Data Protection Regulation [PE16], with particular attention to the principles of data minimisation, purpose limitation and the right to erasure.

The mechanisms that store only public keys and impose no cross origin correlation rate *High*, the mechanisms that mandate a federation profile and delegate deletion to an external provider *Complex*.

6.4 Consolidated Evaluation Matrix

The table 6.2 presents the complete comparative evaluation across all six authentication mechanisms and the sixteen evaluation criteria defined in the framework of Section 6.3, integrating the pior literature done in the Section 2.1 with the theoretical analysis of the Chapter 4 and the prototype validation of the Section 6.2, in the following table.

Criterion	Password	OTP	Federated	MFA	WebAuthn (HW)	WebAuthn (Passkey)
Security						
Phishing resistance	None	None	Partial	Partial	High	High
Replay prevention	Low	High	Moderate	High	High	High
Server breach exposure	High	High	Moderate	Moderate	Minimal	Minimal
MITM resistance	Low	Low	Moderate	Moderate	High	High
Clone detection	None	None	None	None	High	None
Platform account risk	N/A	N/A	High	Moderate	Low	Moderate
Usability						
Interaction cost	Low	Moderate	Low	Moderate	Low	Low
Accessibility	High	Moderate	Moderate	Moderate	Moderate	Moderate
Account recovery	Simple	Moderate	Delegated	Moderate	Complex	Improved
Deployment						
Server impl. cost	Low	Moderate	Moderate	Moderate	High	High
Third party dep.	None	None	High	Moderate	Low	Moderate
Browser support	Univ.	High	Univ.	High	High	High
Legacy compat.	Univ.	High	Moderate	High	Moderate	Moderate
Privacy						
Server data min.	Hash+PII	Sec+PII	Profile	Hash+PII	PK only	PK only
Anonymity support	None	None	None	None	High	Moderate
GDPR alignment	Mod.	Mod.	Complex	Mod.	High	High

Table 6.2: Full comparative evaluation matrix.

7 Conclusions

7.1 Synthesis of Findings

The analysis and validation of the propotype presented in this work provide a coherent overview of the security trade-offs introduced by WebAuthn passkey synchronisation.

Three findings should be highlighted as important.

First, the core cryptographic security properties of WebAuthn, that are **phishing resistance**, **MITM resistance**, **replay attack prevention** and **elimination of server side shared secrets**, are preserved in the synchronised passkey model.

From the perspective of the relying party, a passkey assertion is indistinguishable from a hardware-bound assertion and is equally immune to the attack classes that dominate real-world credential compromise, namely phishing and credential stuffing.

The prototype test suite provides empirical confirmation of this, because the tests T3–T6 demonstrate that none of the standard network level attacks succeed against the verified ceremony logic.

Second, the security of a synchronised passkey does not depend only on the cryptographic properties of WebAuthn, but also the security of the user's Apple ID, Google Account or Microsoft Account.
For users whose platform account is protected by a password and OTP via SMS, the effective security of their **passkeys may be no stronger than the weakest link in that chain**.

This is a structural limitation of the synchronisation model, not an implementation defect, and it's essentially invisible to the relying party.
The comparative matrix (Table 6.2) reflects this by assigning a "Moderate" platform account risk rating to passkeys, in contrast to the "Low" rating for hardware-bound Level 2 credentials.

Third, **clone detection is unavailable for synchronised passkeys**.
This is an intentional design choice made to ensure usability across devices, maintaining a globally consistent counter across synchronised copies would be architecturally incompatible with offline capability and low latency authentication.

The relying parties for which clone detection is a security requirement, such as high value financial or government applications, cannot achieve it with synchronised passkeys and must require hardware-bound credentials.
The prototype's T7–T8 test pair precisely demonstrates this distinction.

Taken together, these findings suggest that **passkeys represent a significant improvement over conventional authentication mechanisms** for the majority of web application contexts, although they don't match the security guarantees offered by hardware-bound credentials in high security environments.

The question of which deployment is appropriate is therefore depends on the context.

7.2 Applicability to Different Deployment Contexts

Consumer applications. The platform passkeys represent the most appropriate authentication mechanism for consumer facing services as of 2026.

They deliver WebAuthn's structural phishing resistance and server secret elimination with a user experience comparable to federated login and passkey synchronisation significantly reduces the account recovery problem that was the primary barrier to WebAuthn adoption in consumer contexts [Lya+20].

The platform account risk is real but is comparable to the risk profile of federated login with the same provider and the overall level of security is substantially higher than password based or OTP authentication.

High security contexts (banking, healthcare, government). The WebAuthn with hardware-bound UV required authenticators remains the appropriate mechanism for contexts requiring NIST AAL3 or equivalent.

The synchronised passkeys are classified at AAL2 by NIST SP 800-63B-4 [Tem+25], the absence of clone detection and the platform account dependency disqualify them from high security contexts where the cost of account compromise is severe.
The deployment cost is justified by the severity of the threat.

Enterprise contexts. Managed hardware security keys deployed via mobile device management (MDM) satisfy enterprise identity and access management requirements and provide attestation based device provenance.

The test T11 of the prototype demonstrates the RP side policy mechanism for enforcing hardware-only credential registration.

The recovery procedures in enterprise contexts are manageable via help desk workflows with identity reverification.

Deployments with limited resources or legacy systems. Where the WebAuthn adoption is not immediately possilbe, the TOTP based MFA with hardware token second factors provides the most pragmatic security improvement over passwords, offering replay resistance at low deployment cost.

The phishing resistance requires hardware-bound second factors, therefore the use of SMS based OTPs should be avoided for high value accounts.

7.3 Conclusions

This work has presented a analysis and evaluation of the security implications of passkey synchronisation and the cross device credential portability within the WebAuthn Level 3 architecture, validated through a working passwordless authentication prototype.

The principal conclusions are as follows.
The **WebAuthn Level 3 passkeys preserve the core cryptographic security properties of the Level 2 specification**, that are phishing resistance, replay prevention, MITM

resistance and server secret elimination, while introducing the platform account as a new attack vector.

The **effective security of a synchronised passkey is linked by the security of the platform account managing synchronisation**, a dependency that is invisible to relying parties and not captured in the WebAuthn protocol layer.

The clone detection, a security property available in hardware-bound deployments, is structurally unavailable for synchronised passkeys due to the impossibility of maintaining globally consistent counters across synchronised copies.

The **relying parties that require clone detection must enforce hardware-bound credential** policy at registration time, as demonstrated in the prototype.

For the majority of deployment contexts, **synchronised passkeys represent a clear security improvement over conventional password based, OTPs and federated authentication mechanisms**.

The account recovery, historically the primary barrier to WebAuthn adoption, is substantially improved by the platform passkey ecosystem, but at the cost of plaform provider dependency.

The remaining obstacles to wider deployment are operational and trust based rather than cryptographic.

Platform provider concentration, the absence of formally verified synchronisation architectures and the absence of a universally standardization recovery mechanism are the most significant open problems identified by this analysis.

8 Future Work and Research Directions

The results and limitations identified in this project point to several promising directions for future research and engineering.

8.1 Standardization Account Recovery for Passkeys

While passkey synchronisation substantially improve account recovery relative to hardware only deployment, recovery remains incompletely standardised and provider dependent.

A user who loses all devices registred to their platform account and loses access to that account will suffer a total loss of their credentials.

The research into cryptographic recovery protocols, such as Threshold Secret Sharing (TSS) based account recovery or decentralised identity (DIDs) based recovery [Spo+22], which could preserve the security properties of WebAuthn without reintroducing less secure authentication methods, represents a highly valuable area of research.

In short, one possible direction for research would be to identify the best protocol for performing a secure account recovery in the event of total data loss.

8.2 Platform Providers Trust Concentration and Governance

The concentration of passkey synchronisation infrastructure in three hyperscale platform providers creates systemic dependencies with implications for competition law, national security and regulatory governance.

A future research would be about a rigorous third party audit of the encryption architecture of each platform's passkey synchronisation implementation, including the key derivation hierarchy, the Hardware Security Module (HSM) access model, the cloud storage encryption scheme and the device enrollment protocol [Uni26] [App26b].

However, we believe that the best option would be to the development of an open, proveidor neutral passkey synchronisation protocol would significantly reduce the systemic trust concentration identified in this work.

8.3 Post-Quantum Cryptography Integration

The dominant WebAuthn algorithms (ES256/ECDSA P-256, EdDSA/Ed25519) are based on elliptic curve discrete logarithm problems, which are vulnerable to Shor's quantum algorithm [Sho94].

The NIST post-quantum cryptography standardisation process [ST25] has finalised ML-DSA (Dilithium) and SLH-DSA (SPHINCS+) as primary post-quantum signature standards.

Further research is needed into the integration of post-quantum signature algorithms into the FIDO2 ecosystem, including the practical limitations imposed by authenticator hardware (key and signature sizes, signing latency on constrained devices), the migration path for existing credentials and the timeline for preparing for the quantum challenge in the commercial hardware security key market.

8.4 Accessibility Engineering for Passkeys

The biometric as primary passkey authentication flows may present barriers for users with certain motor, visual or neurological differences.

If the passkeys are to be the future of authentication methods, we must ensure that everyone can use them.

The systematic accessibility audits of passkey enrollment and authentication interfaces across major platforms, evaluated with assistive technology users are needed to identify these barriers.

The development of design guidelines for inclusive passkey implementations that offer equivalent security guarantees via accessible interaction modalities, sucha as PIN entry, hardware key button activation or voice confirmation, would support the deployment of passkeys in public sector and healthcare contexts where accessibility is a legal and ethical requirement.

In other words, another area for future research could be how to mitigate or minimise these barriers in order to ensure widespread use for everyone.

9 Glossary of Acronyms

AAGUID Authenticator Attestation GUID.

AAL Authenticator Assurance Level (NIST SP 800-63B-4).

BLE Bluetooth Low Energy.

CBOR Concise Binary Object Representation.

COSE CBOR Object Signing and Encryption.

CTAP Client-to-Authenticator Protocol (FIDO Alliance).

ECDSA Elliptic Curve Digital Signature Algorithm.

FIDO Fast IDentity Online (Alliance).

HOTP HMAC-Based One-Time Password.

HSM Hardware Security Module.

IDP Identity Provider.

JWT JSON Web Token.

MDS3 FIDO Metadata Service version 3.

MFA Multi Factor Authentication.

MITM Man-in-the-Middle (attack).

NIST National Institute of Standards and Technology.

OIDC OpenID Connect.

OTP One-Time Password.

PII Personally Identifiable Information.

RP Relying Party.

TOTP Time-Based One-Time Password

TPM Trusted Platform Module.

TTL Time To Live.

UV User Verification.

UP User Presence.

W3C World Wide Web Consortium.

Bibliography

[All22a] FIDO Alliance. *Client to Authenticator Protocol (CTAP) – FIDO2 CTAP 2.1 Specification*. Proposed Standard. 2022. URL: https://fidoalliance.org/specs/fido-v2.1-ps-20210615/fido-client-to-authenticator-protocol-v2.1-ps-errata-20220621.html.

[All22b] FIDO Alliance. *Multi-Device FIDO Credentials: Technical Considerations and Guidance*. 2022. URL: https://fidoalliance.org/white-paper-multi-device-fido-credentials/.

[All23] FIDO Alliance. *Passkeys.dev: Developer Resources for Implementing Passkeys*. Device Support, Accessed 2026-03-23. 2023. URL: https://passkeys.dev/device-support/.

[All25] FIDO Alliance. *Client to Authenticator Protocol (CTAP) – FIDO2 CTAP 2.2 Specification*. Proposed Standard. 2025. URL: https://fidoalliance.org/specs/fido-v2.2-ps-20250714/fido-client-to-authenticator-protocol-v2.2-ps-20250714.html.

[All26] FIDO Alliance. *Client to Authenticator Protocol (CTAP) – FIDO2 CTAP 2.3 Specification*. Proposed Standard. 2026. URL: https://fidoalliance.org/specs/fido-v2.3-ps-20260226/fido-client-to-authenticator-protocol-v2.3-ps-20260226.html.

[App26a] Apple Inc. *iCloud Keychain Security Overview*. https://support.apple.com/guide/security/icloud-keychain-security-overview-sec1c89c6f3b. Apple Platform Security Documentation, Accessed 2026-03-13. 2026.

[App26b] Apple Inc. *Secure Enclave*. https://support.apple.com/guide/security/secure-enclave-sec59b0b31ff. Apple Platform Security Documentation, Accessed 2026-03-13. 2026.

[Bir22] Arnar Birgisson. *Security of Passkeys in the Google Password Manager*. Google Online Security Blog, Accessed 2026-03-13. 2022. URL: https://security.googleblog.com/2022/10/SecurityofPasskeysintheGooglePasswordManager.html.

[Bon+12] Joseph Bonneau et al. "The Quest to Replace Passwords: A Framework for Comparative Evaluation of Web Authentication Schemes". In: *IEEE Symposium on Security and Privacy*. 2012, pp. 553–567. DOI: 10.1109/SP.2012.44.

[Cor23] Microsoft Corporation. *Support for passkeys in Windows*. Microsoft Learn Documentation, Accessed 2026-03-14. 2023. URL: https://learn.microsoft.com/en-us/windows/security/identity-protection/passkeys/.

[Das+18] Anupam Das et al. "The Tangled Web of Password Reuse". In: *Network and Distributed System Security Symposium (NDSS)*. 2018. DOI: 10.14722/ndss.2014.23357.

[Dev26] Alexis Deveria. *Can I Use: Web Authentication API*. Accessed 2026-03-23. 2026. URL: https://caniuse.com/webauthn.

[Eur22] European Parliament and Council of the European Union. *Directive (EU) 2022/2555 on Measures for a High Common Level of Cybersecurity Across the Union (NIS2 Directive)*. Directive 2022/2555. European Union, 2022. URL: https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:02022L2555-20221227.

[Eur24] European Commission. *Regulation on a European Digital Identity Framework (eIDAS 2.0)*. Regulation (proposal and amendments to Regulation (EU) No 910/2014). European Union, 2024. URL: https://digital-strategy.ec.europa.eu/en/policies/eidas-regulation.

[Far+20] Florian M. Farke et al. "'You still use the password after all' - Exploring FIDO2/WebAuthn Usability from User Experience and Adoption Perspectives". In: *Symposium on Usable Privacy and Security (SOUPS)*. USENIX Association, 2020. URL: https://www.usenix.org/system/files/soups2020-farke.pdf.

[Har12] Dick Hardt. *The OAuth 2.0 Authorization Framework*. RFC 6749. IETF, 2012. DOI: 10.17487/RFC6749.

[Inc23] Apple Inc. *iCloud Keychain Overview*. 2023. URL: https://support.apple.com/en-us/102651.

[JMP22] Michael B. Jones, Nandakumar Mehta, and Michael Prorock. *CBOR Object Signing and Encryption (COSE): Initial Algorithms*. RFC 9053. IETF, 2022. DOI: 10.17487/RFC9053.

[Lab25] Duo Labs. *py_webauthn: Pythonic WebAuthn*. GitHub repository, accessed 2025-12-22. 2025. URL: https://github.com/duo-labs/py_webauthn.

[Lya+20] Sanam Ghorbani Lyastani et al. "Is FIDO2 the Kingslayer of User Authentication? A Comparative Usability Study of FIDO2 Passwordless Authentication". In: *IEEE Symposium on Security and Privacy*. 2020, pp. 268–285. DOI: 10.1109/SP40000.2020.00047.

[MRa+05] David M'Raihi et al. *HOTP: An HMAC-Based One-Time Password Algorithm*. RFC 4226. IETF, 2005. DOI: 10.17487/RFC4226.

[MRa+11] David M'Raihi et al. *TOTP: Time-Based One-Time Password Algorithm*. RFC 6238. IETF, 2011. DOI: 10.17487/RFC6238.

[OWU21] Kentrell Owens, Shuman Wang, and Blase Ur. "User Perceptions of the Usability and Security of Smartphones as FIDO2 Roaming Authenticators". In: *Symposium on Usable Privacy and Security (SOUPS)*. USENIX Association, 2021. URL: https://www.usenix.org/system/files/soups2021-owens.pdf.

[PE16] European Parliament and Council of the European Union. *Regulation (EU) 2016/679 – General Data Protection Regulation*. Tech. rep. EU, 2016. URL: http://data.europa.eu/eli/reg/2016/679/2016-05-04.

[Per18] Trevor Perrin. *The Noise Protocol Framework*. 2018. URL: https://noiseprotocol.org/noise.html.

[Pis26] Deepak Babu Piskala. *Spec-Driven Development: From Code to Contract in the Age of AI Coding Assistants*. 2026. arXiv: 2602.00180 [cs.SE]. URL: https://arxiv.org/abs/2602.00180.

[Sak+23] Nat Sakimura et al. *OpenID Connect Core 1.0 Incorporating Errata Set 2*. 2023. URL: https://openid.net/specs/openid-connect-core-1_0.html.

[Sch17] Jim Schaad. *CBOR Object Signing and Encryption (COSE)*. RFC 8152. IETF, 2017. DOI: 10.17487/RFC8152.

[Sho94] Peter W. Shor. "Algorithms for Quantum Computation: Discrete Logarithms and Factoring". In: *IEEE Symposium on Foundations of Computer Science (FOCS)* (1994), pp. 124–134. DOI: 10.1109/SFCS.1994.365700.

[Spo+22] Manu Sporny et al. *Decentralized Identifiers (DIDs) v1.0*. W3C Recommendation. 2022. URL: https://www.w3.org/TR/did-core/.

[ST25] National Institute of Standards and Technology. *Post-Quantum Cryptography Standardization*. NIST IR 8545. NIST, 2025. DOI: 10.6028/NIST.IR.8545.

[Tem+25] D Temoshok et al. *Digital Identity Guidelines: Authentication and Authenticator Management*. NIST Special Publication 800-63B-4. National Institute of Standards and Technology, 2025. DOI: 10.6028/NIST.SP.800-63b-4.

[Uni26] Unit 42, Palo Alto Networks. *Google Authenticator: The Hidden Mechanisms of Passwordless Authentication*. Published 2026. Accessed: 2026-04-13. Feb. 2026. URL: https://unit42.paloaltonetworks.com/passwordless-authentication/.

[Ver25] Verizon. *2025 Data Breach Investigations Report*. Technical Report. Verizon, 2025. URL: https://www.verizon.com/business/resources/T49/reports/2025-dbir-data-breach-investigations-report.pdf.

[W3C19] World Wide Web Consortium (W3C). *Web Authentication: An API for Accessing Public Key Credentials – Level 1*. W3C Recommendation. 2019. URL: https://www.w3.org/TR/webauthn-1/.

[W3C21] World Wide Web Consortium (W3C). *Web Authentication: An API for Accessing Public Key Credentials – Level 2*. W3C Recommendation. 2021. URL: https://www.w3.org/TR/webauthn-2/.

[W3C26] World Wide Web Consortium (W3C). *Web Authentication: An API for Accessing Public Key Credentials – Level 3*. W3C Candidate Recommendation Snapshot. 2026. URL: https://www.w3.org/TR/webauthn-3/.

[Wür+23] Leon Würsching et al. "FIDO2 the Rescue? Platform vs. Roaming Authentication on Smartphones". In: Association for Computing Machinery, 2023. ISBN: 9781450394215. DOI: 10.1145/3544548.3580993. URL: https://doi.org/10.1145/3544548.3580993.